

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Bộ Môn PYTHON



BÁO CÁO MÔN PYTHON

Sinh viên thực hiện:

Đinh Việt Dũng : B23DCKH031

Nguyễn Vĩnh Tùng : B23DCKH130

Nguyễn Vĩnh Trung : B23DCCN862

Lớp : D23 - 046

Mã nhóm : 02

Hà Nội - 2025

MỤC LỤC

Câu I.	2
Phần I.1: Thu thập dữ liệu thống kê của tất cả các cầu thủ có số phút thi đấu nhiều hơn 90 phút tại giải bóng đá ngoại hạng Anh mùa 2024-2025.	2
Phần I.2 - Thu Thập Giá Trị Chuyển Nhượng Cầu Thủ	12
Câu III	27
Phần III.1.1: Phân tích dữ liệu thống kê giải Ngoại Hạng Anh 2024 -2025	27
Phần III.1.2: Phân tích phong độ đội bóng:	33
Phần III.2: Đề xuất và Xây dựng phương pháp định giá cầu thủ:	39
Câu IV.	42
Phần IV.1: Sử dụng thuật toán K-mean để phân loại cầu thủ:	42
Phần IV.2: Theo bạn thì nên phân loại cầu thủ thành bao nhiêu nhóm? Nhận xét về kết quả.	49
Phần IV.3: Sử dụng thuật toán PCA, giảm số chiều dữ liệu xuống 2 chiều, 3 chiều, vẽ hình scatter plot phân cụm	53

Câu I.

Phần I.1: Thu thập dữ liệu thống kê của tất cả các cầu thủ có số phút thi đấu nhiều hơn 90 phút tại giải bóng đá ngoại hạng Anh mùa 2024-2025.

1. Mô tả đề bài

Đề bài yêu cầu xây dựng một chương trình thu thập dữ liệu thống kê của tất cả các cầu thủ tham gia thi đấu tại giải bóng đá Ngoại hạng Anh mùa giải 2024 – 2025, với điều kiện chỉ giữ lại những cầu thủ có tổng số phút thi đấu lớn hơn 90 phút. Dữ liệu phải được lấy từ trang web thống kê bóng đá FBref tại địa chỉ <https://fbref.com/en/>.

Toàn bộ kết quả sau khi thu thập và xử lý phải được lưu trữ vào một bảng trong cơ sở dữ liệu SQLite. Cấu trúc bảng được quy định theo nguyên tắc: mỗi cột tương ứng với một chỉ số thống kê của cầu thủ (ví dụ như số trận ra sân, số bàn thắng, số đường chuyền, số phút thi đấu, các chỉ số phòng ngự, tấn công, kiểm soát bóng, v.v.). Đối với bất kỳ chỉ số nào không tồn tại, không được cung cấp cho một cầu thủ cụ thể hoặc không áp dụng (ví dụ thủ môn không có một số chỉ số dành riêng cho cầu thủ tấn công), giá trị trong bảng phải được ghi là chuỗi “N/a”.

2. Phân tích yêu cầu

a. Dữ liệu phạm vi thu thập

- Dữ liệu phải được lấy cho toàn bộ cầu thủ thi đấu tại giải Ngoại hạng Anh mùa 2024 – 2025, không chỉ một đội bóng. Điều này đồng nghĩa với việc chương trình phải truy cập trang tổng hợp của giải, sau đó làm việc với các bảng thống kê cấp độ giải đấu (comps/9/2024-2025) thay vì chỉ tập trung vào một câu lạc bộ đơn lẻ.

b. Điều kiện lọc

- Chỉ các cầu thủ có tổng số phút thi đấu lớn hơn 90 phút mới được đưa vào kết quả cuối cùng. Do đó, trong dữ liệu thu thập bắt buộc phải có cột liên quan đến số phút thi đấu (ví dụ “Minutes”, “Min”, “Min Played”, tùy theo cách FBref đặt tên), và sau khi gom dữ liệu cho từng cầu thủ, chương trình phải áp dụng bộ lọc này trước khi ghi vào cơ sở dữ liệu.

c. Nguồn dữ liệu và cách trích xuất

FBref tổ chức dữ liệu dưới dạng các bảng HTML trên trang web. Một số bảng có thể được hiển thị trực tiếp, một số khác có thể được đặt trong các đoạn comment HTML nhằm hạn chế việc trích xuất tự động. Vì vậy, chương trình cần có khả năng: tải trang, truy cập đúng phần chứa bảng, đọc mã HTML, phân tích cấu trúc bảng và trích xuất nội dung từng hàng, từng cột.

d. Mô hình lưu trữ

- Kết quả cuối cùng phải được lưu vào một bảng SQLite, trong đó:
 - Mỗi hàng phải tương ứng với một câu hỏi
 - Mỗi cột tương ứng với một chỉ số thống kê
 - Giá trị rỗng, thiếu, không áp dụng phải chuẩn hóa thành “N/a” để đảm bảo tính thống nhất, tránh để trống hoặc kiểu dữ liệu không đồng nhất

e. Kỹ thuật cài đặt

- Để đáp ứng yêu cầu tự động thu thập dữ liệu từ một trang web động, chương trình cần sử dụng các công cụ như Selenium, Undetected_chromedriver hoặc trình điều khiển trình duyệt tương tự để mô phỏng thao tác của người dùng, kết hợp với thư viện phân tích HTML như BeautifulSoup để xử lý nội dung trang. Khi đã có dữ liệu ở dạng bảng, có thể sử dụng pandas để đưa về dạng DataFrame và sqlite3 để lưu lại vào cơ sở dữ liệu SQLite.

3. Thư viện yêu cầu

- **Undetected_chromedriver:** Khởi tạo Chrome driver có khả năng tránh bị phát hiện là bot.
- **Selenium:** Tự động hóa trình duyệt, tương tác với trang web.
- **BeautifulSoup4:** Parse và phân tích HTML.
- **Pandas:** Xử lý vào thao tác dữ liệu dạng bảng
- **SQLite3:** Lưu trữ dữ liệu vào database
- **Collections:** Tạo dictionary tự động khởi tạo giá trị mặc định

4. Ý tưởng và hướng giải quyết

Ý tưởng tổng quát của bài toán là xây dựng một pipeline tự động gồm các bước: truy cập trang web thống kê, trích xuất dữ liệu từ nhiều bảng khác nhau, hợp nhất dữ liệu theo từng cầu thủ, lọc theo số phút thi đấu rồi lưu kết quả cuối cùng vào cơ sở dữ liệu SQLite với cấu trúc cột tương ứng với từng chỉ số.

Cụ thể, có thể triển khai theo hướng sau:

a. Khởi tạo và cấu hình trình duyệt tự động

- Sử dụng **undetected_chromedriver** kết hợp với **Selenium** để khởi tạo một phiên bản trình duyệt Chrome có thể vượt qua một số cơ chế phát hiện bot. Thiết lập thời gian chờ tải trang, phóng to cửa sổ và chèn các khoảng dừng ngẫu nhiên để mô phỏng hành vi người dùng thực tế.

b. Duyệt các trang thống kê theo từng nhóm chỉ số

- Từ URL gốc của mùa giải Ngoại hạng Anh 2024 – 2025, kết hợp với các đường dẫn con tương ứng với từng loại bảng thống kê (standard, shooting, passing, passing types, gca, defense, possession, playing_time, misc, goalkeeping, goalkeeping_adv). Đối với mỗi loại bảng, chương trình tải trang, chờ phần tử chứa bảng xuất hiện, sau đó trích xuất nội dung HTML của container tương ứng.

c. Phân tích cấu trúc bảng và trích xuất dữ liệu

- Sử dụng **BeautifulSoup** để phân tích **HTML**, xác định hàng tiêu đề (**thead**) và nội dung (**tbody**). Tên cột được lấy từ thuộc tính data-stat hoặc từ nội dung của thẻ **th**. Mỗi hàng dữ liệu được chuyển thành một dict, trong đó các trường quan trọng như tên cầu thủ, câu lạc bộ, quốc tịch, tuổi, vị trí được chuẩn hóa lại tên. Với các bảng khác nhau, có thể thêm tiền tố vào tên chỉ số để phân biệt nguồn gốc (ví dụ: shooting__, passing__, defense__), đảm bảo không trùng tên cột khi hợp nhất.

d. Hợp nhất dữ liệu theo từng cầu thủ

- Sử dụng một cấu trúc dữ liệu dạng **defaultdict(dict)** với khóa là cặp (tên cầu thủ, câu lạc bộ). Mỗi lần đọc xong một bảng thống kê, chương trình lần lượt hợp nhất các chỉ số mới vào bản ghi tương ứng của cầu thủ. Các thông tin cơ bản lấy từ bảng standard được xem là chuẩn và chỉ bị ghi đè khi còn thiếu.

e. Lọc theo số phút thi đấu

- Sau khi có đầy đủ dữ liệu, chương trình kiểm tra các trường liên quan đến số phút thi đấu, chuyển về số nguyên và chỉ giữ lại những cầu thủ có tổng số phút thi đấu lớn hơn 90. Đây là bước ràng lọc quan trọng nhằm loại bỏ những trường hợp ít ra sân, giúp dữ liệu đầu ra ổn định, có ý nghĩa hơn cho phân tích thống kê.

f. Chuẩn hóa và lưu dữ liệu vào SQLite

- Từ cấu trúc dict của tất cả cầu thủ, chương trình chuyển sang **DataFrame** của pandas, xử lý các cột trùng lặp và đổi tên các cột cho thống nhất (ví dụ đổi squad thành club). Toàn bộ giá trị rỗng, **NaN** hoặc không hợp lệ được chuyển thành chuỗi "**N/a**" đúng yêu cầu đề bài. Cuối cùng, chương trình kết nối tới cơ sở dữ liệu SQLite, tạo mới bảng kết quả với mỗi cột tương ứng một chỉ số và chèn lần lượt từng hàng dữ liệu vào.

5. Cấu Trúc Chương Trình

- Chương trình được tổ chức thành các hàm riêng biệt, mỗi hàm đảm nhận một giai đoạn trong quy trình xử lý dữ liệu từ FBref.

a. Nhóm hàm cấu hình và anti-detection

- **build_driver()**
- Khởi tạo Chrome driver với undetected_chromedriver để tránh bị phát hiện là bot.
- Quy trình chính:
 - Thử khởi tạo với Chrome version 141, nếu thất bại thì auto-detect version hiện tại
 - Set timeout 90 giây cho page load và script execution
 - Maximize cửa sổ browser để đảm bảo tất cả elements được render đầy đủ
 - Kết quả: Trả về Chrome driver instance sẵn sàng sử dụng
- **human_pause(min_delay, max_delay)**
- Giúp trình duyệt mô phỏng hành vi người dùng thực bằng cách tạo khoảng dừng ngẫu nhiên.
- Quy trình:
 - Tạo delay ngẫu nhiên từ 0.7 đến 1.6 giây (mặc định)
 - Sử dụng random.uniform() để tránh pattern đều đặn
 - Mục đích: Tránh bị FBref phát hiện là bot và rate limit/IP ban

b. Hàm tải và thu thập HTML

- **extract_table_html(driver, container_id, table_id)**
- Là hàm trung tâm của giai đoạn Extract, trích xuất HTML của bảng thống kê từ trang web.
- Quy trình chính:
 - Tìm container element theo container_id bằng Selenium

- Lấy innerHTML của container
- Parse HTML bằng BeautifulSoup với parser lxml
- Tìm bảng theo table_id - trước tiên tìm trực tiếp trong DOM
- Nếu không tìm thấy, tìm trong HTML comments <!-- --> (FBref ẩn một số bảng trong comments)
- Trả về HTML string của bảng hoàn chỉnh

● Kết quả: HTML string của bảng hoặc None nếu không tìm thấy

c. Hàm trích xuất và phân tích dữ liệu bảng

- parse_table_html(table_html, page_type)
- Sử dụng BeautifulSoup để phân tích mã HTML và chuyển đổi thành dữ liệu có cấu trúc.
- Quy trình:
 - Validate input: kiểm tra table_html có hợp lệ
 - Trích xuất headers từ dòng cuối cùng của <thead> (FBref sử dụng multi-level headers)
 - Ưu tiên lấy giá trị từ attribute data-stat thay vì text hiển thị
 - Duyệt qua từng row trong <tbody>, bỏ qua các header rows được chèn giữa dữ liệu
 - Xử lý đặc biệt cho cột "player" (lấy cả tên và link) và "squad/team" (normalize thành "squad")
 - Gọi _process_stat_cell() để xử lý các cells còn lại
 - Kết quả: List of dictionaries, mỗi dictionary đại diện cho một cầu thủ với đầy đủ thông kê
- _process_stat_cell(row_data, key, value, page_type)
- Chuẩn hóa tên cột và giá trị cho từng cell dữ liệu.
- Quy trình:
 - Định nghĩa mapping cho các cột cơ bản: born → birth_year, pos → position, nationality → nation

- Chỉ normalize khi crawl từ bảng "standard" để tránh tạo cột trùng lặp (ví dụ: shooting__age, passing__age)
- Các cột khác giữ nguyên tên gốc
- Kết quả: row_data được cập nhật với tên cột đã chuẩn hóa
- d. Nhóm hàm xử lý và làm sạch dữ liệu**
- **merge_player_stats(master_data, new_rows, page_type)**
- Hợp nhất dữ liệu từ 11 bảng thống kê khác nhau thành một dataset thống nhất.
- Quy trình:
 - Sử dụng tuple (player, squad) làm unique key để phân biệt cầu thủ chuyển đội giữa mùa
 - Sử dụng defaultdict(dict) để tự động tạo entry mới khi cần
 - Merge stats từ các page khác nhau với naming convention:
 - Bảng "standard": goals, assists, minutes
 - Bảng khác: shooting__shots, passing__passes_completed, defense__tackles
 - Replace empty values thành "N/a" để phân biệt với giá trị 0
- Kết quả: master_data chứa tất cả stats của tất cả cầu thủ, mỗi cầu thủ là một dictionary
- **filter_by_minutes(player_data, threshold)**
- Lọc dữ liệu chỉ giữ các cầu thủ có số phút thi đấu lớn hơn ngưỡng (mặc định 90 phút).
- Quy trình:
 - Tìm cột minutes trong 3 cột khả dĩ với thứ tự ưu tiên:
 - minutes (từ bảng standard - cầu thủ thường)
 - playing_time__minutes (từ bảng playing_time - chi tiết hơn)
 - goalkeeping__gk_minutes (từ bảng goalkeeper - chỉ có cho thủ môn)
 - Convert string thành integer, loại bỏ dấu phẩy nếu có ("1,234" → 1234)
 - Lọc chỉ giữ cầu thủ có minutes > threshold

- Kết quả: Dictionary đã được lọc, đảm bảo dữ liệu đại diện cho các cầu thủ ra sân thường xuyên
- **clean_duplicate_columns(dataframe)**
- Làm sạch DataFrame bằng cách loại bỏ các cột có dữ liệu trùng lặp.
- Quy trình:
 - Định nghĩa các nhóm cột có khả năng trùng lặp: games/matches, age, position, nation...
 - Tìm tất cả cột trong DataFrame khớp với mỗi nhóm
 - Giữ lại cột ưu tiên (primary_key), xóa các cột còn lại
 - Exclude các cột có pattern đặc biệt: "90", "per_90", "starts", "subs" (không phải duplicate)
- Kết quả: DataFrame đã loại bỏ duplicate columns, giảm từ ~270 cột xuống ~180 cột

e. Hàm lưu trữ dữ liệu

- **save_to_sqlite(dataframe, db_path, table_name)**
- Lưu DataFrame đã xử lý vào SQLite database với cấu trúc tối ưu.
- Quy trình:
 - Gọi clean_duplicate_columns() để làm sạch lần cuối
 - Rename cột "squad" → "club" (thuật ngữ phổ biến hơn)
 - Sắp xếp cột theo thứ tự logic:
 - Base columns ở đầu: player, club, age, birth_year, nation, position
 - Stat columns theo category: standard, shooting, passing, defense...
 - DROP TABLE cũ nếu tồn tại
 - CREATE TABLE mới với tất cả cột type TEXT
 - Insert data bằng executemany() (nhanh hơn nhiều lần so với loop)
 - Replace NULL/NaN/None thành "N/a" để đồng nhất dữ liệu
- Kết quả: File SQLite với bảng player_stats_cleaned chứa dữ liệu đã chuẩn hóa

f. Hàm main()

- Là trái tim của chương trình, điều phối toàn bộ quy trình ETL (Extract, Transform, Load).

- Quy trình chính:
 - Khởi tạo Chrome driver với build_driver()
 - Khởi tạo defaultdict(dict) để lưu trữ dữ liệu tất cả cầu thủ
 - Duyệt qua 11 trang thống kê trong STAT_PAGES:
 - Gọi driver.get(url) để navigate đến trang
 - Wait cho container element load bằng WebDriverWait
 - Gọi extract_table_html() để lấy HTML bảng
 - Gọi parse_table_html() để parse thành structured data
 - Gọi merge_player_stats() để gộp vào master dataset
 - Random pause để tránh rate limit
 - Gọi filter_by_minutes() để lọc cầu thủ có > 90 phút
 - Convert sang pandas DataFrame
 - Drop cột player_link (không cần thiết cho analysis)
 - Gọi save_to_sqlite() để lưu vào database
 - Finally block: đóng driver dù có lỗi hay không để clean up resources
- Kết quả: File SQLite fbref_pl_2024_2025_full.sqlite chứa ~500 cầu thủ với ~180 cột thống kê

6. Khó khăn và giải pháp

a. FBref Anti-Bot Protection

+ Khó khăn

- Cloudflare Protection: FBref sử dụng hệ thống Cloudflare để chống bot tự động
- Selenium thường bị phát hiện: Trình duyệt thông thường qua Selenium dễ bị detect và block
- Rate Limiting: Các request gửi quá nhanh sẽ bị server từ chối

+ Hướng giải quyết

- Sử dụng undetected_chromedriver thay vì Selenium thường:

- Thư viện `undetected_chromedriver` được thiết kế đặc biệt để tránh bị phát hiện
- Loại bỏ các dấu hiệu nhận dạng bot như `webdriver flags`, `automation extensions`
- Giả lập hành vi trình duyệt thật một cách tự nhiên hơn
- Áp dụng random delays giữa các requests:
 - Tạo khoảng dừng ngẫu nhiên từ 0.7 đến 1.6 giây giữa mỗi request
 - Mô phỏng hành vi người dùng thật (không ai click quá nhanh đều đặn)
 - Tránh pattern recognition của hệ thống anti-bot
- Không sử dụng headless mode:
 - Chạy browser với giao diện đầy đủ (không ẩn)
 - Headless mode thường là dấu hiệu rõ ràng của bot
 - Browser hiển thị giúp tăng độ tin cậy
- Set timeout hợp lý:
 - Cho phép trang web đủ thời gian load và render (90 giây)
 - Tránh timeout sớm gây lỗi không cần thiết

b. Dữ Liệu Ẩn Trong HTML Comments

+ Khó khăn

- Bảng không hiển thị trực tiếp: Một số bảng thống kê được FBref ẩn trong HTML comments `<!-- -->`
- Chỉ load khi user click: Bảng chỉ được uncomment khi người dùng click vào tab tương ứng
- Selenium không thấy: Nếu chỉ parse DOM thông thường sẽ bị thiếu dữ liệu (30-40%)

+ Hướng giải quyết

- Lấy toàn bộ innerHTML của container:
 - Sử dụng Selenium để lấy innerHTML thay vì chỉ lấy các elements hiển thị
 - innerHTML chứa cả phần comment nên không bỏ sót dữ liệu
- Parse cả HTML comments:
 - Sử dụng BeautifulSoup để tìm tất cả HTML comments trong source code
 - Comments có dạng `<!-- <table>...</table> -->`
 - FBref ẩn bảng trong comments để giảm tải trang ban đầu
- Tìm kiếm 2 lần (fallback mechanism):
 - Bước 1: Tìm bảng trực tiếp trong DOM (các bảng đã được render)
 - Bước 2: Nếu không tìm thấy, tìm trong comments (các bảng ẩn)
 - Đảm bảo thu thập được tất cả bảng

- Parse lại HTML từ comment:
 - Khi tìm thấy table trong comment, parse lại content của comment
 - Chuyển string HTML thành BeautifulSoup object để xử lý tiếp
- c. Cột Trùng Lặp Từ Nhiều Bảng**
- + **Khó khăn**
 - 11 bảng, mỗi bảng có cột lặp: Mỗi bảng thống kê đều có các cột: age, position, team, matches...
 - Dữ liệu trùng 100%: Nếu không xử lý sẽ có 11 cột age, 11 cột position với data giống hệt nhau
 - DataFrame phình to: ~270 cột thay vì ~180 cột cần thiết
 - Tốn memory và khó phân tích
- + **Hướng giải quyết**
 - Normalize cột cơ bản ngay khi crawl:
 - Xác định các cột cơ bản: age, position, team, nation, birth_year...
 - Tạo mapping chuẩn cho các tên cột biến thể (ví dụ: "born" → "birth_year", "pos" → "position")
 - Chỉ lấy giá trị từ bảng "standard" cho các cột cơ bản
 - Các bảng khác sẽ không tạo cột trùng lặp này
 - Thêm prefix cho cột của mỗi bảng:
 - Bảng "standard": giữ nguyên tên cột (goals, assists, minutes...)
 - Các bảng khác: thêm prefix theo loại (shooting__shots, passing__passes, defense__tackles...)
 - Dễ phân biệt cột nào từ bảng nào
 - Định nghĩa nhóm cột duplicate:
 - Tạo dictionary các nhóm cột có khả năng trùng lặp
 - Mỗi nhóm có 1 cột ưu tiên (primary key)
 - Ví dụ: nhóm "age" gồm [age, shooting__age, passing__age...]
 - Phát hiện và xóa cột trùng:
 - Tìm tất cả cột trong DataFrame khớp với từng nhóm
 - Giữ lại cột ưu tiên (primary key)
 - Xóa tất cả các cột còn lại trong nhóm
 - Exclude các cột có pattern đặc biệt (90, per_90, starts, subs)
 - Thực hiện 2 lần:
 - Lần 1: Khi merge dữ liệu từ 11 bảng
 - Lần 2: Trước khi save vào SQLite (clean lần cuối)
- d. Multi-Level Table Headers**

+ **Khó khăn**

- Header có nhiều cấp: FBref sử dụng header 2-3 cấp (Performance > Goals, Expected > xG...)
- Không biết lấy dòng nào: Khó xác định dòng nào chứa tên cột thật
- Colspan gây nhầm lẫn: Các cell với colspan làm lệch số lượng cột
- Text hiển thị khác data thật: Text hiển thị là "Gls" nhưng tên cột thực tế là "goals"

+ **Hướng giải quyết**

- Luôn lấy dòng cuối cùng của thead:
 - Thead có nhiều rows, nhưng chỉ row cuối chứa tên cột chi tiết
 - Các row trên chỉ là grouping/category
 - Parse tất cả rows và chọn row cuối (index = -1)
- Ưu tiên attribute data-stat:
 - FBref lưu tên cột thật trong attribute data-stat của thẻ <th>
 - Text hiển thị cho người đọc (viết tắt, có định dạng)
 - Attribute cho máy đọc (tên đầy đủ, không format)
 - Luôn lấy data-stat trước, nếu không có mới lấy text
- Fallback mechanism:
 - Nếu không có data-stat attribute
 - Lấy text content của <th> và normalize
 - Đảm bảo luôn có tên cột dù format thế nào
- Xử lý colspan:
 - Không cần xử lý đặc biệt vì chỉ lấy row cuối
 - Row cuối không có colspan, mỗi cell = 1 cột
 - Các row trên có colspan nhưng không dùng

Phần I.2 - Thu Thập Giá Trị Chuyển Nhượng Cầu Thủ

1. Mô tả đề bài

Sau khi hoàn thành phần I.1 với dữ liệu thống kê chi tiết về các cầu thủ Premier League mùa 2024-2025, phần I.2 yêu cầu bổ sung thêm thông tin về giá trị chuyển nhượng (market value) của từng cầu thủ vào cơ sở dữ liệu.

Cụ thể, chương trình cần:

- Lấy danh sách tất cả cầu thủ từ bảng `player_stats_cleaned` đã tạo ở phần I.1

- Thu thập giá trị chuyển nhượng hiện tại của từng cầu thủ từ website footballtransfers.com
- Lưu kết quả vào một bảng mới player_transfer_values_2024_2025 trong cùng database SQLite
- Đảm bảo xử lý các trường hợp: tên cầu thủ có dấu, bí danh, không tìm thấy thông tin

Kết quả cuối cùng phải bao gồm các trường: tên cầu thủ, giá trị chuyển nhượng (market value), cầu lạc bộ hiện tại, và ngày cập nhật thông tin.

2. Phân tích yêu cầu

a. Nguồn dữ liệu đầu vào

Dữ liệu đầu vào là danh sách cầu thủ từ bảng player_stats_cleaned trong file SQLite fbref_pl_2024_2025_full.sqlite. Chương trình cần query để lấy danh sách tên cầu thủ duy nhất và lặp qua từng cầu thủ để thu thập thông tin giá trị chuyển nhượng.

b. Nguồn dữ liệu thu thập

- Website footballtransfers.com cung cấp thông tin giá trị chuyển nhượng của cầu thủ. Cấu trúc URL có dạng:
 - URL trực tiếp:
`https://www.footballtransfers.com/en/players/{player-slug}`
 - URL tìm kiếm:
<https://www.footballtransfers.com/en/players?q={player-name}>
- Trong đó player-slug là tên cầu thủ được chuẩn hóa

c. Xử lý tên cầu thủ

- Tên cầu thủ trong database có thể:
 - Có dấu (ví dụ: "Emiliano Martínez", "João Pedro")
 - Có nhiều phần (ví dụ: "Mohamed Salah")
 - Có alias khác nhau trên website (ví dụ: "Bruno Fernandes" có thể là "bruno-fernandes-1", "bruno-fernandes-2")

Do đó, chương trình cần:

- Chuẩn hóa tên thành slug (bỏ dấu, lowercase, thay space bằng -)
- Thử nhiều variants (thêm -1, -2, -3 vào cuối slug)
- Sử dụng search API nếu không tìm thấy trực tiếp
- Log các alias để tái sử dụng

d. Thông tin cần thu thập

Từ trang profile của cầu thủ, cần extract:

- Market Value: Giá trị chuyển nhượng (định dạng: €10.0m, €50.0k...)
- Club: Câu lạc bộ hiện tại
- Last Update: Ngày cập nhật thông tin gần nhất

e. Xử lý trường hợp đặc biệt

- Không tìm thấy cầu thủ: Ghi giá trị "N/a" cho tất cả các trường
- Redirect: Website có thể redirect từ URL này sang URL khác, cần follow redirect và log lại
- Timeout: Request quá lâu, cần retry với timeout cao hơn
- Cầu thủ trùng tên: Sử dụng search results để chọn đúng cầu thủ

f. Mô hình lưu trữ

Tạo bảng mới `player_transfer_values_2024_2025` với cấu trúc:

Cột	Kiểu dữ liệu	Mô tả
player	TEXT	Tên cầu thủ
market_value	TEXT	Giá trị chuyển nhượng (€)
club	TEXT	Câu lạc bộ hiện tại
last_update	TEXT	Ngày cập nhật

Tất cả cột đều dùng TEXT để đơn giản hóa, giá trị thiếu ghi "N/a".

2. Thư viện yêu cầu

- sqlite3: Kết nối và thao tác với SQLite database
- requests: Gửi HTTP requests để tải trang web
- BeautifulSoup4: Parse và phân tích HTML
- unidecode: Chuyển đổi ký tự Unicode sang ASCII
- re: Xử lý regular expressions để chuẩn hóa tên
- urllib.parse: Encode URL parameters cho search query

3. Ý tưởng và hướng giải quyết

- Tổng quan

Xây dựng một pipeline tự động gồm các bước: lấy danh sách cầu thủ từ database, chuẩn hóa tên, thử nhiều URL variants để tìm thông tin, parse HTML để extract market value, và lưu kết quả vào database.

- Chi tiết triển khai

a. Khởi tạo database và bảng kết quả

- Kết nối tới database SQLite hiện có
- Tạo bảng `player_transfer_values_2024_2025` nếu chưa tồn tại
- Xóa dữ liệu cũ để đảm bảo chạy fresh mỗi lần

b. Chuẩn hóa tên cầu thủ thành slug

- Sử dụng unicodecode để:
 - Chuyển Unicode → ASCII (Martínez → Martinez)
 - Lowercase tất cả ký tự
 - Loại bỏ ký tự đặc biệt (giữ chữ cái, số, space, dấu gạch ngang)
 - Thay space bằng dấu gạch ngang
 - Kết quả: "Mohamed Salah" → "mohamed-salah"

c. Sử dụng search API khi không tìm thấy trực tiếp

- Nếu tất cả variants đều thất bại:
 - Gửi request tới
`https://www.footballtransfers.com/en/players?q={player-name}`
 - Parse HTML để tìm link đầu tiên trong search results
 - Extract href và crawl trang đó
 - Hoặc lấy luôn thông tin từ search results (có market value preview)

d. Parse HTML để extract thông tin

Sử dụng BeautifulSoup với CSS selectors:

- Market Value: `.player-value` `.player-tag` hoặc `.playerInfo-value`
- Club: `.playerInfoTop-bar__club` hoặc `.playerInfoTop-bar__team`
- Last Update: Tìm text chứa "Last update" và extract phần sau dấu :

Validate: Market value phải bắt đầu bằng € để đảm bảo trang đúng.

e. Xử lý timeout và retry

- Default timeout: 15 giây
- Nếu timeout, retry 1 lần với timeout 20 giây
- Nếu vẫn fail, ghi "N/a" và tiếp tục cầu thủ khác
- Không dừng chương trình vì một cầu thủ

f. Lưu từng record vào database

Sau mỗi lần crawl thành công một cầu thủ:

- Insert ngay vào database (không đợi hết)
- Commit transaction để đảm bảo dữ liệu được lưu
- Tránh mất dữ liệu nếu chương trình bị gián đoạn

4. Cấu trúc chương trình

- Chương trình được tổ chức thành các hàm chức năng riêng biệt:

a. Nhóm hàm database

- **init_db()**
 - Tạo bảng player_transfer_values_2024_2025 nếu chưa tồn tại
 - Định nghĩa schema với 4 cột: player, market_value, club, last_update
 - Tất cả cột type TEXT
- **clear_table()**
 - Xóa toàn bộ dữ liệu cũ trong bảng
 - Thực hiện trước khi bắt đầu crawl mới
 - Đảm bảo không có dữ liệu duplicate
- **insert_record(player, value, club, date)**
 - Insert một record vào database
 - Commit ngay sau khi insert
 - Không batch để tránh mất data nếu crash

b. Nhóm hàm xử lý tên và URL

- **make_slug(name)**
 - Chuyển tên Unicode sang ASCII bằng unidecode()
 - Lowercase toàn bộ
 - Loại bỏ ký tự đặc biệt (chỉ giữ a-z, 0-9, space, dash)
 - Replace space bằng -
 - Kết quả: slug chuẩn để tạo URL
- **log_alias(original, alias_url)**
 - Ghi lại mapping giữa tên gốc và URL thực tế
 - Append vào file alias_detected.txt
 - Hữu ích cho debug và tối ưu hóa

c. Nhóm hàm crawl dữ liệu

- **crawl_page(url)**
 - Gửi GET request với User-Agent header

- Follow redirects (allow_redirects=True)
- Nếu URL redirect, log alias
- Parse HTML bằng BeautifulSoup
- Extract market value, club, last_update bằng CSS selectors
- Validate: market value phải bắt đầu bằng €
- Return tuple (value, club, date) hoặc None nếu fail
- Xử lý timeout: retry 1 lần với timeout cao hơn
- **crawl_from_search_results(soup)**
 - Parse search results page
 - Tìm item đầu tiên trong listing
 - Extract market value và club trực tiếp từ search preview
 - Return tuple (value, club, "Approx") nếu tìm thấy
 - Return None nếu không tìm thấy
- **fetch_player_value(player)**
 - Hàm chính để lấy thông tin của 1 cầu thủ
 - Quy trình:
 - Tạo slug từ tên cầu thủ
 - Thử 4 URL variants: base, -1, -2, -3
 - Nếu tìm thấy, return ngay
 - Nếu không, search qua search API
 - Parse link đầu tiên trong search results
 - Crawl trang đó hoặc lấy từ search preview
 - Nếu vẫn không có, return ("N/a", "N/a", "N/a")

d. Hàm main()

- **main()**
 - Khởi tạo database và clear bảng cũ
 - Query danh sách cầu thủ **DISTINCT** từ player_stats_cleaned
 - In tổng số cầu thủ cần crawl
 - Khởi tạo set seen để tránh crawl duplicate
 - Loop qua từng cầu thủ:
 - Check xem đã crawl chưa (trong seen)
 - In progress: [1/500] Mohamed Salah
 - Gọi **fetch_player_value()**
 - Insert vào database

- Add vào seen
- Catch exception và ghi "N/a" nếu có lỗi
- In thông báo hoàn tất

5. Khó khăn và giải pháp

Tên Cầu Thủ Có Dấu và Biến Thể

+ Khó khăn:

- Tên cầu thủ trong database có ký tự Unicode (dấu): "João Félix", "Emiliano Martínez"
- URL slug không dùng Unicode, cần chuyển sang ASCII
- Nhiều cách chuyển đổi khác nhau có thể dẫn đến URL sai

+ Hướng giải quyết:

- Sử dụng thư viện unidecode:
 - Thư viện được thiết kế chuyên biệt cho việc chuyển Unicode → ASCII
 - Xử lý đúng hầu hết các ký tự đặc biệt của các ngôn ngữ
 - Ví dụ: "José" → "Jose", "Müller" → "Muller"
- Chuẩn hóa thống nhất:
 - Lowercase toàn bộ ký tự
 - Loại bỏ tất cả ký tự không phải chữ cái, số, space, dash
 - Replace space bằng - để tạo URL-friendly slug
- Xử lý edge cases:
 - Xóa space thừa ở đầu/cuối
 - Xóa nhiều dash liên tiếp thành 1 dash
 - Đảm bảo slug không bắt đầu/kết thúc bằng dash

Cầu Thủ Trùng Tên (Duplicate Names)

+ Khó khăn:

- Nhiều cầu thủ có cùng tên: "Gabriel", "Pedro", "Bruno Fernandes"
- Website thêm suffix -1, -2, -3 để phân biệt
- Không biết trước suffix nào đúng cho cầu thủ cụ thể

+ Hướng giải quyết:

- Thử tuần tự nhiều variants:
 - Tạo list URL với các suffix: base, -1, -2, -3
 - Loop qua từng URL theo thứ tự
 - Dừng ngay khi tìm thấy trang hợp lệ (có market value)
- Validate kết quả:
 - Kiểm tra market value phải bắt đầu bằng €

- Nếu không có €, coi như trang không hợp lệ
 - Continue thử URL tiếp theo
- Early stopping:
 - Không cần thử tất cả 4 URLs nếu đã tìm thấy
 - Return ngay khi có kết quả hợp lệ
 - Tiết kiệm thời gian và requests
- Log alias để tái sử dụng:
 - Ghi lại mapping: tên gốc → URL thực tế
 - Có thể build dictionary để optimize lần chạy sau
 - Giúp debug khi có vấn đề
- Kết quả đạt được:
 - Tìm đúng hầu hết câu thủ dù trùng tên
 - Không bỏ sót do chọn sai variant
 - Có thể optimize dần theo thời gian

Request Timeout và Network Errors

- + **Khó khăn:**
 - Request có thể timeout (server chậm, mạng kém)
 - Connection errors ngẫu nhiên
 - Server có thể trả về HTTP error codes (500, 503...)
- + **Hướng giải quyết:**
 - Set timeout hợp lý:
 - Default timeout: 15 giây (đủ cho hầu hết cases)
 - Không quá thấp (dễ timeout)
 - Không quá cao (chờ lâu khi thực sự có vấn đề)
 - Exception handling:
 - Wrap mọi network call trong try-except
 - Catch Exception rộng để cover tất cả cases
 - Log error nhưng không dừng program

Kết quả đạt được:

- Program chạy được dù network không ổn định
- Tự động retry khi có thể
- Không mất toàn bộ data vì 1 request fail

Câu II.

- **Tổng quan**

Part 2 cung cấp các công cụ để truy vấn và phân tích dữ liệu Premier League đã thu thập ở Part 1:

- Part 2_1: RESTful API (FastAPI) để truy vấn database
- Part 2_2: CLI tool để tra cứu và export CSV

- **Mục tiêu**

- Xây dựng REST API với FastAPI để expose dữ liệu
- Hỗ trợ tìm kiếm theo tên cầu thủ và câu lạc bộ
- Asynchronous database operations với aiosqlite
- CLI tool để query API và export CSV
- Auto-generate API documentation (Swagger/ReDoc)

Phần 2.1: FastAPI REST API

Cấu trúc thư mục

```
Part 2_1/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── core/
│   │   └── config.py
│   ├── routers/
│   │   └── players.py
│   ├── crud/
│   │   └── player.py
│   ├── db/
│   │   └── sqlite.py
│   └── schemas/
│       └── types.py
├── fbref_pl_2024_2025_full.sqlite
└── requirements.txt
```

Công nghệ sử dụng

```
fastapi>=0.104.0
uvicorn[standard]>=0.24.0
aiosqlite>=0.19.0
```

API Endpoints

1. GET /players

Tìm kiếm cầu thủ theo tên (LIKE search)

- Request:

GET /players?name=haaland

- Response:

```
{
  "player": "Erling Haaland",
  "squad": "Manchester City",
  "age": "24",
  "birth_year": "2000",
  "nation": "NOR",
  "position": "FW",
  "standard__games": "31",
  "standard__goals": "27",
  "standard__assists": "5",
  "standard__minutes": "2593",
  ...
}
```

- Parameters:

- name: Tên cầu thủ (case-insensitive, partial match)

- Features:

- Trả về tất cả kết quả phù hợp
- Case-insensitive search
- Partial matching: "salah" → "Mohamed Salah"
- Trả về đầy đủ 231 cột

2. GET /players/by-club/{club}

Lấy toàn bộ cầu thủ của một câu lạc bộ

- Request:

GET /players/by-club/Arsenal?exact=true

- Parameters:

- Club (path, required): Tên câu lạc bộ
- Exact (query, default=true): true = Exact match, false = LIKE search

- Features:
 - Trả về toàn đội hình (thường 20-30 cầu thủ)
 - Exact match mặc định
 - Option cho fuzzy search

Chi tiết kỹ thuật

- Quản lý vòng đời

```

1  @asynccontextmanager
2  async def lifespan(app: FastAPI):
3      async with aisqlite.connect(str(DB_FILE)) as conn:
4          conn.row_factory = aisqlite.Row
5          await ensure_source_table(conn)
6          await create_indexes(conn)
7          print(f"Kết nối database đã sẵn sàng: {DB_FILE}")
8
9      yield
10
11     await close_db()
12     print("Đã đóng kết nối database")

```

- Lợi ích:
 - Kiểm tra database availability khi startup
 - Tạo indexes tự động (player, squad)
 - Graceful shutdown

Hoạt động Cơ sở dữ liệu Bất đồng bộ

```

1  async def get_db() -> AsyncGenerator[aisqlite.Connection, None]:
2      async with aisqlite.connect(str(DB_FILE)) as conn:
3          conn.row_factory = aisqlite.Row
4          yield conn

```

- Lợi ích:
 - Connection per request (thread-safe)
 - Tự động close connection
 - Row factory cho dict output
- Tiêm phụ thuộc

```
1 @router.get("", summary="Tìm cầu thủ theo tên")
2 async def by_name(
3     name: str = Query(..., description="Tên cầu thủ, ví dụ: 'haaland'"),
4     db: aioredis.Connection = Depends(get_db),
5 ) -> List[Dict[str, Any]]:
6     """Tìm kiếm cầu thủ theo tên (trả về tất cả kết quả phù hợp)"""
7     rows = await fetch_by_name(db, name=name)
8     if not rows:
9         raise HTTPException(status_code=404, detail=f"Không tìm thấy cầu thủ phù hợp với tên '{name}'")
10    return rows
```

- Lợi ích:
 - Chia nhỏ thành phần
 - Dễ dàng kiểm thử
 - Làm sạch code
- Tài liệu được sinh tự động
- FastAPI tự động tạo interactive docs:
 - Swagger UI: <http://127.0.0.1:8000/docs> - Interactive API testing
 - ReDoc: <http://127.0.0.1:8000/redoc> - Clean, readable

Phần 2.2: Nslookup

- Chức năng

Script lookup.py cung cấp command-line interface để:

- Query FastAPI từ terminal
- Hiển thị kết quả dạng table
- Export kết quả ra CSV file

- Sử dụng

- **Tìm kiếm theo tên cầu thủ**

```
python lookup.py --name "Bukayo Saka"
```

Output:

```
[
  {
    "player": "Bukayo Saka",
    "club": "Arsenal",
    "age": "22",
    "birth_year": "2001",
```

```
"nation": "engENG",  
    ....  
    }  
]  
  
CSV đã lưu: bukayo_saka.csv
```

- **Tìm kiếm theo câu lạc bộ**

python lookup.py --club "Arsenal"

Output:

```
[  
  {  
    "player": "Riccardo Calafiori",  
    "club": "Arsenal",  
    "age": "22",  
    "birth_year": "2002",  
    "nation": "itITA",  
    "position":  
    "DF",...  
  },  
  {  
    "player": "Kai Havertz",  
    "club": "Arsenal",  
    "age": "25",  
    "birth_year": "1999",  
    "nation": "deGER",  
    "position": "FW,MF",  
    "shooting__goals":
```

```
"9",
}.....
}
]

CSV đã lưu: arsenal.csv
```

Tính năng

- Tự động đổi tên
 - Input: --name "Mohamed Salah"
 - Output: mohamed_salah.csv
 - Slugify: normalize Unicode, replace spaces với underscore
- Xử lý lỗi
 - Connection error: Gợi ý chạy server
 - API error: Hiển thị status code và detail
 - Missing data: Thông báo rõ ràng
- Hỗ trợ Unicode

with open(filename, "w", newline="", encoding="utf-8-sig") as f:

- UTF-8 with BOM cho Excel compatibility
- Hỗ trợ tên có dấu, ký tự đặc biệt

• So sánh FastAPI và Flask

1. So sánh tổng quan

Tiêu chí	FastAPI	Flask
Hiệu suất	Rất cao (xử lý đồng thời)	Thấp hơn (xử lý tuần tự)
Tài liệu tự động	Tự động (Swagger/ReDoc)	Phải viết thủ công
Kiểm tra kiểu dữ liệu	Có (báo lỗi tự động)	Không
Kiểm tra giá trị	Tự động	Phải code thủ công
Xử lý đồng thời	Có sẵn async/await	Cần thư viện thêm

2. Chi tiết các ưu điểm của FastAPI

a. Xử lý đồng thời (Async/Await) - Hiệu suất vượt trội

- FastAPI

```
1 async def fetch_by_name(conn: aiomysql.Connection, name: str) -> List[Dict]:
2     sql = f"SELECT * FROM [{SOURCE_TABLE}] WHERE LOWER(player) LIKE LOWER(?)"
3     async with conn.execute(sql, (f"%{name}%",)) as cursor:
4         rows = await cursor.fetchall()
5         return [dict(row) for row in rows]
```

```
1 @router.get("", summary="Tìm cầu thủ theo tên")
2 async def by_name(
3     name: str = Query(..., description="Tên cầu thủ, ví dụ: 'haaland'"),
4     db: aiomysql.Connection = Depends(get_db),
5 ) -> List[Dict[str, Any]]:
6     """Tìm kiếm cầu thủ theo tên (trả về tất cả kết quả phù hợp)"""
7     rows = await fetch_by_name(db, name=name)
8     if not rows:
9         raise HTTPException(status_code=404, detail=f"Không tìm thấy cầu thủ phù hợp với tên '{name}'.")
10    return rows
```

- Lợi ích:

- FastAPI: Xử lý nhiều yêu cầu cùng lúc, không bị chặn
- Flask: Mỗi yêu cầu phải đợi nhau, bị chặn
- Hiệu suất: FastAPI nhanh hơn 2-3 lần khi có nhiều người dùng cùng lúc

b. Tài liệu tự động

- FastAPI:

- Swagger UI: <http://127.0.0.1:8000/docs> - Giao diện để test API
- ReDoc: <http://127.0.0.1:8000/redoc> - Tài liệu đẹp để đọc
- OpenAPI schema tự động sinh ra
- Có thể test API ngay trên trình duyệt

- Flask: Phải viết tài liệu riêng (Word, Markdown)

- Hoặc dùng Flask-RESTX (cài thêm, phức tạp)
- Phải bảo trì 2 nơi: code + tài liệu

3. Kiểm tra kiểu dữ liệu và giá trị tự động

- FastAPI:

```
1 @router.get("/by-club/{club}", summary="Lấy toàn bộ cầu thủ thuộc CLB")
2 async def players_of_club(
3     club: str = Path(..., description="Tên câu lạc bộ, ví dụ: 'Arsenal'"),
4     exact: bool = Query(True, description="True = khớp chính xác; False = LIKE search"),
5     db: aiomysqlite.Connection = Depends(get_db),
6 ) -> List[Dict[str, Any]]:
7     """Lấy toàn bộ chỉ số của tất cả cầu thủ thuộc CLB"""
8     rows = await fetch_by_club(db, club=club, exact=exact)
9     if not rows:
10         raise HTTPException(status_code=404, detail=f"Không tìm thấy cầu thủ thuộc CLB '{club}'.")
11     return rows
```

- Lợi ích:

- Khai báo kiểu: Editor tự động gợi ý, phát hiện lỗi trước khi chạy
- Tự động kiểm tra: exact phải là true/false, tự động chuyển đổi
- Thông báo lỗi tự động: {"detail": "value is not a valid boolean"}
- Editor biết cấu trúc dữ liệu trả về

4. Hệ thống tiêm phụ thuộc (Dependency Injection)

- FastAPI:

```
1 @router.get("", summary="Tìm cầu thủ theo tên")
2 async def by_name(
3     name: str = Query(..., description="Tên cầu thủ, ví dụ: 'haaland'"),
4     db: aiomysqlite.Connection = Depends(get_db),
5 ) -> List[Dict[str, Any]]:
```

- Lợi ích:

- Tự động chèn kết nối database
- Tự động dọn dẹp (đóng kết nối)
- Dễ test (có thể giả lập phụ thuộc)
- Code sạch, không lặp lại

Câu III

Phần III.1.1: Phân tích dữ liệu thống kê giải Ngoại Hạng Anh 2024 -2025

1. Phân tích yêu cầu đề bài:

. **Mục tiêu chính:**

- Chương trình được thiết kế để thực hiện phân tích thống kê tổng hợp dữ liệu cầu thủ từ một cơ sở dữ liệu SQLite (fbef_pl_2024_2025_full.sqlite)

a. **Yêu cầu:**

- **Tính toán thống kê đội:**
 - Tính **trung bình có trọng số** (weight mean) dựa trên số phút thi đấu (minutes) của cầu thủ. Điều này đảm bảo cầu thủ thi đấu nhiều hơn sẽ có ảnh hưởng lớn hơn đến chỉ số trung bình của đội.
 - Tính **trung vị** (median) và **độ lệch chuẩn** (std) tiêu chuẩn.
 - **Xử lý dữ liệu:** Chương trình phải tự động phát hiện các cột dữ liệu quan trọng, xử lý các giá trị thiếu (missing data) và làm sạch dữ liệu trước khi tính toán.
 - **Kết quả:** Xuất bảng thống kê tổng hợp (bao gồm 3 chỉ số cho mỗi chỉ số gốc) ra một file CSV duy nhất (team_metric_summary.csv) để phục vụ cho các phân tích sâu hơn.

2. **Thư viện sử dụng**

Thư viện	Vai trò chính
Sqlite3	Kết nối và thực thi truy vấn SQL để đọc dữ liệu từ file cơ sở dữ liệu SQLite.
Pandas(pd)	Thư viện cốt lõi: Đọc dữ liệu từ SQL vào DataFrame, thực hiện thao tác groupby, áp dụng (apply) các hàm thống kê tùy chỉnh, và xuất kết quả ra file CSV.
Numpy (np)	Hỗ trợ tính toán số học, đặc biệt là các hàm thống kê an toàn (như np.nansum, np.nanmean, np.nanmedian) để xử lý các giá trị thiếu (NaN).
Pathlib (path)	Quản lý đường dẫn file một cách linh hoạt và độc lập với hệ điều hành, giúp xác định vị trí file database và file CSV đầu ra.

Typing(list, optional)	Cung cấp các gợi ý kiểu (type hints) để làm cho code rõ ràng và dễ bảo trì hơn.
-------------------------------	---

3. Phương pháp và quy trình thực hiện:

a. Phương pháp:

- Chương trình áp dụng phương pháp phân tích dữ liệu theo nhóm bằng Pandas:
- **Tính trung bình theo trọng số (Weighted Mean):** Đây là phương pháp thống kê chính xác hơn khi tổng hợp dữ liệu cầu thủ. Thay vì dùng `.agg(["mean"])` (coi mỗi cầu thủ là như nhau) thì ta sẽ sử dụng `.apply(minutes_weighted_stats)`. Hàm này sẽ tính toán một trọng số (**w**) cho mỗi cầu thủ dựa trên tỷ lệ số phút thi đấu của họ so với tổng số phút của cả đội. Trung bình của mỗi chỉ số sau đó được tính bằng `np.nansum(chỉ_số * trọng_số)`.
- **Phát hiện cột tự động:** Code dùng hàm `detect_cols` để tự động tìm các cột “club”, “player” và “minutes” giúp code linh hoạt hơn nếu cấu trúc của bảng thay đổi.
- **Làm sạch dữ liệu linh hoạt:** Hàm `to_numeric_safe` xử lý nhiều định dạng dữ liệu thiếu (N/A) và tự động chuyển đổi dấu (,) về dấu (.) cho các số thập phân.

b. Quy trình thực hiện:

- **Khởi tạo (setup):** xác định các hằng số và đường dẫn file (Database, bảng, file CSV đầu ra)
- **Kết nối và tải (extract):** Kết nối đến file SQLite. Kiểm tra xem file có tồn tại không. Đọc toàn bộ dữ liệu từ bảng `player_stats_cleaned` vào một DataFrame của Pandas.
- **Tiền xử lý (transform):**
 - Gọi `detect_cols` để xác định tên cột chính xác cho “club”, “player” và “minutes”.
 - Gọi `coerce_numeric` để lặp qua tất cả các cột, áp dụng cho `to_numeric_safe` để làm sạch và ép kiểu dữ liệu sang dạng số (float), đồng thời bỏ qua các cột văn bản (như “player”, “nation”).
 - **Tính toán thống kê (calcuculate):**

- Kết quả từ **.apply()** được tập hợp thành một DataFrame **summary** mới.
- DataFrame này được làm tròn (**.round()**) và lưu vào file **team_metric_summary.csv** bằng cách sử dụng dấu chấm phẩy (;) làm dấu phân tách và dấu phẩy (,) cho số thập phân.

4. Mô tả các hàm :

a. Hàm làm sạch dữ liệu:

```
def to_numeric_safe(s: pd.Series) -> pd.Series:
    """Chuyển đổi cột sang số, xử lý các giá trị missing và dấu phẩy thập phân"""
    if s.dtype == object:
        s = s.replace(list(NA_STRINGS), np.nan)
        s = s.apply(lambda x: str(x).replace(',', '.')) if isinstance(x, str) else x
    return pd.to_numeric(s, errors="coerce")
```

- **Nhiệm vụ:** Chuyển đổi một **pd.Series** (một cột) sang kiểu số một cách an toàn.
- **Mô tả:** Hàm này thay thế tất cả các chuỗi được định nghĩa trong **NA_STRINGS** (như "N/A", "None", "-") bằng **np.nan**. Nó cũng thay thế dấu phẩy thập phân (,) bằng dấu chấm (.) để đảm bảo **pd.to_numeric** hoạt động chính xác. Tham số **errors="coerce"** biến mọi giá trị không thể chuyển đổi thành **NaN**.

b. Hàm hỗ trợ:

- Hàm **detect_col**

```
def detect_cols(df: pd.DataFrame,
               team_col: Optional[str], player_col: Optional[str], minutes_col: Optional[str]):
    """Tự động detect tên cột nếu chưa được chỉ định"""
    if team_col is None:
        if "team" in df.columns:
            team_col = "team"
        elif "club" in df.columns:
            team_col = "club"
        elif "squad" in df.columns:
            team_col = "squad"
    if player_col is None:
        player_col = "player" if "player" in df.columns else None
    if minutes_col is None:
        cands = [c for c in df.columns if "minutes" in c.lower() and "90" not in c.lower()]
        minutes_col = cands[0] if cands else None
    return team_col, player_col, minutes_col
```

- **Nhiệm vụ:** Tự động tìm tên các cột quan trọng.

- **Mô tả:** Nếu tên cột không được cung cấp, hàm này sẽ tự tìm các ứng viên (ví dụ: "team", "club", hoặc "squad" cho cột đội) trong danh sách cột của DataFrame.

- Hàm `coerce_numeric(df, exclude):`

```
def coerce_numeric(df: pd.DataFrame, exclude: List[str]) -> List[str]:
    numeric_cols = []
    for c in df.columns:
        if c in exclude:
            continue
        df[c] = to_numeric_safe(df[c])
        if df[c].notna().any():
            numeric_cols.append(c)
    return numeric_cols
```

- **Nhiệm vụ:** Lập và làm sạch tất cả các cột số.
- **Mô tả:** Hàm này duyệt qua mọi cột trong DataFrame, ngoại trừ những cột trong danh sách **exclude**. Nó áp dụng **to_numeric_safe** cho từng cột và trả về một danh sách các tên cột đã được chuyển đổi thành công sang kiểu số.

c. **Hàm tính toán cột lỗi:**

- `minutes_weighted_stats(g, cols, minutes_col):`

```
def minutes_weighted_stats(g: pd.DataFrame, cols: List[str], minutes_col: Optional[str]) -> pd.Series:
    out = {}
    w = None
    if minutes_col and minutes_col in g.columns:
        w = to_numeric_safe(g[minutes_col]).clip(lower=0).replace([np.inf, -np.inf], np.nan).fillna(0.0)
        total = w.sum()
        w = None if (total == 0 or np.isnan(total)) else (w / total)

    for c in cols:
        s = to_numeric_safe(g[c]).astype(float)

        if w is not None:
            mean_val = float(np.nansum(s.values * w.values))
        else:
            mean_val = float(np.nanmean(s.values))

        median_val = float(np.nanmedian(s.values))
        std_val = float(np.nanstd(s.values, ddof=0))

        out[f"{c}_median"] = median_val
        out[f"{c}_mean"] = mean_val
        out[f"{c}_std"] = std_val

    return pd.Series(out)
```

- **Nhiệm vụ:** Đây là hàm quan trọng nhất, thực hiện tính toán thống kê cho một nhóm (một đội).
- **Mô tả:**
 - i. Nhận vào **g** (DataFrame của một đội) và danh sách **cols** (các chỉ số cần tính).
 - ii. Tính toán trọng số **w** bằng cách lấy cột **minutes_col**, làm sạch nó và chia cho tổng số phút (**w / total**) để chuẩn hóa.
 - iii. Lặp qua từng chỉ số **c** trong **cols**
 - **:mean_val (Weighted):** Tính bằng **np.nansum(s.values * w.values)**.
 - **median_val (Standard):** Tính bằng **np.nanmedian(s.values)**.
 - **std_val (Standard):** Tính bằng **np.nanstd(s.values)**.
 - iv. Trả về một **pd.Series** chứa tất cả các kết quả (ví dụ: **age_median**, **age_mean**, **age_std**,...).

d. Hàm điều phối chính:

- Main():

- **Nhiệm vụ:** Điều phối toàn bộ quy trình ETL.
- **Mô tả:** Gọi các hàm theo đúng thứ tự: kết nối DB, đọc dữ liệu, **detect_cols**, **coerce_numeric**, **groupby().apply(minutes_weighted_stats)**, và cuối cùng là **to_csv**.

5. Một số lỗi thường gặp và cách xử lý:

- Lỗi FileNotFoundError (Không tìm thấy Database):

- **Nguyên nhân:** File **fbref_pl_2024_2025_full.sqlite** không tồn tại ở đường dẫn **DB_PATH**.
- **Khắc phục (Đã xử lý):** Code của bạn kiểm tra bằng **if not DB_PATH.exists()** và đưa ra **SystemExit** với thông báo lỗi rõ ràng.

- Lỗi ValueError (Dữ liệu bản khi chuyển sang số):

- **Nguyên nhân:** Các cột số chứa các giá trị như "N/A", "1,234.5" (dùng dấu phẩy kiểu Mỹ) hoặc "12,3" (dùng dấu phẩy thập phân).
- **Khắc phục (Đã xử lý):** Hàm **to_numeric_safe** xử lý triệt để vấn đề này bằng cách thay thế **NA_STRINGS** và chuyển đổi dấu phẩy (,) sang (.), sau đó **pd.to_numeric(errors="coerce")** sẽ chuyển các lỗi còn lại thành **NaN**.

- Lỗi KeyError (Không tìm thấy cột):

- **Nguyên nhân:** Tên cột trong DB khác với tên cột được hard-code (ví dụ: dùng "team" thay vì "club").

- **Khắc phục (Đã xử lý):** Hàm `detect_cols` giúp tìm tên cột một cách linh hoạt. Thêm vào đó, `raise SystemExit` được gọi nếu không tìm thấy cột đội hoặc cầu thủ.
- **Lỗi tính toán trên NaN:**
 - **Nguyên nhân:** Dữ liệu bị thiếu (trống) có thể khiến các phép tính (mean, median) trả về NaN.
 - **Khắc phục (Đã xử lý):** Code sử dụng các hàm của NumPy như `np.nanmean`, `np.nanmedian`, `np.nanstd`, `np.nansum`... Các hàm này tự động bỏ qua (ignore) các giá trị NaN khi tính toán.

6. Kết quả:

- **File CSV đầu ra:** Chương trình tạo thành công file **team_metric_summary.csv**
- **Nội dung:**
 - Mỗi hàng là một câu lạc bộ (CLB).
 - Các cột là các chỉ số thống kê đã được tổng hợp, bao gồm **_mean** (trung bình có trọng số theo phút), **_median** (trung vị) và **_std** (độ lệch chuẩn) cho MỌI chỉ số số học có trong DB.

A	B	C	D	E	F	G	H	I	J	K	L	M	
Club	age_median	age_mean	age_std	birth_year_median	birth_year_mean	birth_year_std	shooting_goals_median	shooting_goals_mean	shooting_goals_std	shooting_goals_per_shot_median	shooting_goals_per_shot_mean	shooting_goals_per_shot_std	shooting_goals_p
Arsenal	25,5	24,91	3,73	1997,5	1998,7	3,86	2,5	1,72	2,84	0,15	0,13	0,13	0,43
Aston Villa	26,0	25,82	3,96	1997,0	1997,85	3,96	1,0	1,83	3,36	0,05	0,08	0,08	0,25
Bournemouth	25,0	24,46	3,96	1999,0	1999,21	3,78	1,0	0,74	3,63	0,07	0,04	0,06	0,25
Brentford	24,0	24,6	3,74	1999,5	1999,09	3,73	0,0	0,42	5,77	0,05	0,03	0,08	0,29
Brighton	23,0	24,43	5,04	2000,0	1999,16	5,08	1,0	0,86	3,12	0,07	0,09	0,09	0,26
Chelsea	23,0	23,29	2,25	2001,0	2000,41	2,49	1,0	0,73	3,59	0,08	0,07	0,12	0,29
Crystal Palace	26,0	25,64	3,31	1998,0	1997,95	3,39	1,0	0,49	3,49	0,05	0,12	0,21	0,19
Everton	25,5	22,73	4,88	1998,5	2000,53	4,93	1,0	0,7	2,45	0,06	0,03	0,1	0,22
Fulham	27,0	26,05	3,81	1996,0	1997,63	3,83	0,0	3,35	3,42	0,02	0,15	0,08	0,17
Ipswich Town	25,5	26,04	3,05	1998,0	1997,63	3,24	0,0	1,4	2,28	0,06	0,08	0,07	0,25
Leicester City	25,0	24,79	4,71	1998,0	1998,57	4,72	0,0	0,51	2,11	0,07	0,05	0,11	0,27
Liverpool	25,0	24,74	3,52	1998,0	1999,02	3,55	1,0	0,14	6,51	0,07	0,01	0,07	0,3
Manchester City	25,0	26,23	4,83	1998,0	1997,55	4,87	1,0	0,93	4,5	0,11	0,17	0,2	0,36
Manchester Utd	24,0	24,44	5,03	1999,0	1999,15	4,92	0,0	0,84	2,3	0,0	0,05	0,07	0,11
Newcastle Utd	26,0	28,01	4,67	1997,0	1995,57	4,57	0,0	0,16	5,09	0,04	0,04	0,09	0,31
Nott'ham Forest	26,0	26,29	3,47	1997,5	1997,17	3,51	1,0	0,68	4,34	0,07	0,04	0,07	0,25
Southampton	25,5	26,4	4,35	1998,5	1997,38	4,44	0,5	0,22	1,04	0,05	0,03	0,09	0,2
Tottenham	24,0	23,86	4,52	1999,0	1999,72	4,47	1,0	0,89	3,26	0,06	0,04	0,08	0,24
West Ham	27,0	27,68	4,99	1997,0	1995,79	4,84	0,0	0,93	3,09	0,04	0,06	0,06	0,2
Wolves	26,5	27,64	3,88	1997,5	1996,12	3,9	1,0	1,2	3,89	0,09	0,11	0,11	0,34

Phần III.1.2: Phân tích phong độ đội bóng:

1. Phân tích yêu cầu đề bài:

a. Mục tiêu chính:

- Mục tiêu của phần này là chuyển đổi dữ liệu thô của cầu thủ (đã được làm sạch ở phần trước) thành các chỉ số ở cấp độ đội (team-level).

b. Yêu cầu:

- Nhiệm vụ 1: xác định đội mạnh nhất

- Tính toán hiệu suất trung bình của mỗi đội cho từng chỉ số.
- Xác định đội bóng có chỉ số trung bình cao nhất ở mỗi chỉ số thống kê đó.
- Lưu kết quả này vào file **metric_leaders.csv**

- **Nhiệm vụ 2: xếp hạng phong độ tổng hợp**

- Sử dụng phương pháp chuẩn hoá **Z-score** để đưa tất cả các chỉ số **_mean** về một thang đo chung.
- Phải xử lý các **chỉ số tiêu cực** (như **card_red**, **misc__fouls**) bằng cách **đảo ngược** điểm **Z-score** của chúng, vì ở các chỉ số này, giá trị *thấp hơn có nghĩa là tốt hơn*.
- Tính điểm **form_index** cuối cùng bằng cách lấy trung bình cộng của tất cả các **Z-score** cho mỗi đội.
- Xuất bảng xếp hạng đã sắp xếp file **team_form_index.csv**

2. Thư viện sử dụng:

Thư viện	Vai trò
Pandas(pd)	Thư viện cốt lõi để đọc CSV, chuẩn hóa Z-Score , xử lý dữ liệu, tìm giá trị lớn nhất (idxmax), tính trung bình (mean) và xuất kết quả ra CSV.
Numpy(np)	Được sử dụng để xử lý các giá trị NaN có thể phát sinh khi chia cho độ lệch chuẩn bằng 0 (nếu tất cả các đội có cùng một chỉ số).
Pathlib(path)	Quản lý đường dẫn file một cách hiệu quả

3. Phương pháp và quy trình thực hiện:

a. Phương pháp

- **Chuẩn hoá Z-score:**

- Công thức $Z = \frac{X - \mu}{\sigma}$ (trong đó μ là trung bình của giải, σ là độ lệch chuẩn của giải).
- Phương pháp này sẽ chuẩn hoá về “**số lần độ lệch chuẩn so với trung bình**”. Ví dụ **Z-score** là **+1.5** có nghĩa là đội đó tốt hơn 1,5 lần độ lệch chuẩn so với trung bình của toàn giải đấu. Điều này cho thấy mức độ “vượt trội” của một đội ở chỉ số đó.

- **Đảo ngược chỉ số tiêu cực:**

- Đây là bước xử lý logic quan trọng. Đối với các chỉ số tiêu cực như thẻ vàng có chỉ số **Z-score** thấp như là **-2.0** tức nghĩa là tốt (ít thẻ phạt hơn trung bình).
- Để đảm bảo các chỉ số này có đóng góp tích cực vào điểm phong độ, code đã đảo ngược lại giá trị **Z-score** của chúng.

b. Quy trình thực hiện:

- **Tải dữ liệu:** Tải dữ liệu từ bảng **team_metric_summary.csv**
- **Chuẩn bị:** Đặt “Club” là index và lọc ra danh sách các cột **_means**.
- **Thực thi nhiệm vụ 1:**
 - Lặp qua danh sách **mean_cols**.
 - Sử dụng **s.idxmax()** để tìm đội dẫn đầu.
 - Lưu kết quả vào **leaders** và xuất ra file **metric_leaders.csv**.
- **Thực thi nhiệm vụ 2:**
 - Tính toán DataFrame **Z** cho tất cả các đội **_mean**.
 - Lặp qua **NEGATIVE_METRICS** và đảo ngược các cột **Z-score** tương ứng.
 - Tính **form-index** bằng cách lấy trung bình theo hàng (**.mean(axis=1)**) của DataFrame **Z**.
 - Lưu bảng xếp hạng đã sắp xếp ra file **team_form_index.csv**.

4. Mô tả các khối logic chính:

a. Khối tải và lọc dữ liệu:

```
df = pd.read_csv(SUMMARY_CSV, sep=';', decimal=',')

if "Club" not in df.columns:
    raise ValueError("File summary phải có cột 'Club'.")
df = df.set_index("Club")

for c in df.columns:
    if c.endswith("_mean"):
        df[c] = pd.to_numeric(df[c], errors="coerce")

mean_cols = [c for c in df.columns if c.endswith("_mean")]
```

- **Mô tả:** Code tải dữ liệu, đặt tên đội làm index và quan trọng nhất là lọc một danh sách **means_cols** chỉ chứa các cột trung bình. Đây là dữ liệu cơ sở cho mọi phân tích sau đó.

b. Khối nhiệm vụ 1: Tìm đội dẫn đầu (metric_leaders.csv)

```

rows = []
for col in sorted(mean_cols):
    s = df[col]
    if s.notna().any():
        top_team = s.idxmax()
        top_val = float(s.loc[top_team])
        rows.append({
            "metric": col[:-5],
            "top_team": top_team,
            "value": round(top_val, ROUND_DIGITS),
        })

leaders = pd.DataFrame(rows)
leaders.to_csv(LEADERS_CSV, index=False, sep=';', decimal=',')

```

- **Mô tả:** Trái tim của khối này là **s.idxmax()**. Hàm này của Pandas trả về tên (**index**) của hàng có giá trị lớn nhất, chính là "đội dẫn đầu" cho chỉ số (cột) col đó.
- Kết quả được lưu vào **file metric_leaders.csv**.

c. **Khối nhiệm vụ 2: Tính chỉ số phong độ Z-score (team_form_index.csv)**

```

Z = (df[mean_cols] - df[mean_cols].mean()) / df[mean_cols].std(ddof=0).replace(0, np.nan)

for m in NEGATIVE_METRICS:
    col = f"{m}_mean"
    if col in Z.columns:
        Z[col] = -Z[col]

fi = Z.mean(axis=1, skipna=True).to_frame("form_index")
fi = fi.sort_values("form_index", ascending=False).round(ROUND_DIGITS)
fi.to_csv(FORMIDX_CSV, sep=';', decimal=',')

```

- Dòng **Z = (df[mean_cols] - df[mean_cols].mean()) / df[mean_cols].std(ddof=0).replace(0, np.nan)** thực hiện phép toán **Z-Score** trên toàn bộ các cột **_mean**.
- Vòng lặp **for m in NEGATIVE_METRICS:** tìm các cột **Z-Score** tương ứng (như **misc__fouls_mean**) và nhân chúng với -1. Điều này đảm bảo các đội phạm ít lỗi sẽ nhận được điểm thưởng, thay vì bị trừ điểm.
- **Z.mean(axis=1)** là lệnh cuối cùng, tính trung bình *theo hàng* (cho từng đội) của tất cả các **Z-score** để tạo ra một con số **form_index** duy nhất, sau đó lưu ra file **team_form_index.csv**.

5. Một số lỗi thường gặp và cách xử lý:

- **Lỗi FileNotFoundError:**
 - **Nguyên nhân:** File đầu vào **team_metric_summary.csv** không tồn tại trong cùng thư mục.
 - **Khắc phục:** Đảm bảo script của Phần III.1.1 đã chạy thành công và tạo ra file này.
- **Lỗi ValueError:**
 - **Nguyên nhân:** File CSV đầu vào không có cột "Club".
 - **Khắc phục (Đã xử lý):** Script đã có lệnh `raise ValueError(...)` để thông báo lỗi rõ ràng nếu cột này bị thiếu.
- **Lỗi chia cho 0 (khi tính Z-Score):**
 - Script đã xử lý khéo léo bằng cách dùng **`.replace(0, np.nan)`** trên mẫu số (std). Phép chia cho **NaN** sẽ tạo ra kết quả **NaN**. Khi tính **`fi = Z.mean(axis=1, skipna=True)`**, các cột NaN này sẽ tự động bị bỏ qua, không ảnh hưởng đến điểm phong độ cuối cùng.

6. Kết quả:

- a. Đội dẫn đầu theo từng chỉ số

	A	B	C
1	metric	top_team	value
2	age	Newcastle Utd	28,01
3	assists	Everton	1,44
4	assists_per90	Everton	0,2
5	birth_year	Everton	2000,53
6	cards_red	Fulham	0,18
7	cards_yellow	Everton	2,82
8	defense__blocked_passes	Bournemouth	7,45
9	defense__blocked_shots	Brighton	4,96
10	defense__blocks	Brighton	11,38
11	defense__challenge_tackles	Arsenal	8,34
12	defense__challenge_tackles_pct	Chelsea	59,05
13	defense__challenges_lost	Nott'ham Forest	11,3
14	defense__challenges	Nott'ham Forest	19,54
15	defense__clearances	Nott'ham Forest	29,7
16	defense__errors	Southampton	1,22
17	defense__interceptions	Nott'ham Forest	8,56
18	defense__minutes_90s	Wolves	8,31
19	defense__tackles_att_3rd	Arsenal	3,02
20	defense__tackles_def_3rd	Nott'ham Forest	11,1
21	defense__tackles_interceptions	Nott'ham Forest	27,36
22	defense__tackles	Nott'ham Forest	18,8
23	defense__tackles_mid_3rd	Nott'ham Forest	6,4
24	defense__tackles_won	Nott'ham Forest	11,02
25	games	Fulham	19,51
26	games_starts	Arsenal	8,47
27	gca__defense	Chelsea	0,18
28	gca__fouled	Leicester City	0,34
29	gca__gca	Brighton	2,85
30	gca__minutes_90s	Wolves	8,31
31	gca__passes_dead	Brentford	0,67
32	gca__passes_live	Everton	2,22
33	gca__per90	Brentford	0,38
34	gca__sca_defense	Bournemouth	0,74
35	gca__sca_fouled	Leicester City	1,18
36	gca__sca	Fulham	20,87
37	gca__sca_passes_dead	Brentford	5,13
38	gca__sca_passes_live	Fulham	15,39

b. Bảng xếp hạng phong độ tổng hợp:

1	Club	form_index
2	Liverpool	0,472
3	Arsenal	0,445
4	Wolves	0,422
5	Brighton	0,406
6	Fulham	0,274
7	Tottenham	0,262
8	Manchester City	0,107
9	Bournemouth	0,101
10	Leicester City	0,073
11	West Ham	0,028
12	Chelsea	-0,004
13	Ipswich Town	-0,016
14	Aston Villa	-0,112
15	Nott'ham Forest	-0,134
16	Brentford	-0,135
17	Manchester Utd	-0,274
18	Newcastle Utd	-0,279
19	Everton	-0,289
20	Southampton	-0,642
21	Crystal Palace	-0,705

Phần III.2: Đề xuất và Xây dựng phương pháp định giá cầu thủ:

1. Phân tích yêu cầu đề bài:

- Mục tiêu của phần này là đề xuất một phương pháp định giá cầu thủ dựa trên các dữ liệu thống kê đã thu thập (từ Câu I.1) và dữ liệu giá trị thị trường (từ Câu I.2).
- Chương trình sau đây là một phương pháp được triển khai để giải quyết bài toán này. Chương trình này xây dựng một mô hình học máy để dự đoán giá trị chuyển nhượng của cầu thủ dựa trên các chỉ số thống kê của họ.

2. Xử lý và giải quyết bài toán: Phương pháp hồi quy tuyến tính

a. Tải và hợp nhất dữ liệu:

- **Tải dữ liệu:** Chương trình kết nối đến CSDL `fbref_pl_2024_2025_full.sqlite` và tải dữ liệu từ hai bảng:
 - `player_stats_cleaned` (chứa thống kê chi tiết).
 - `player_transfer_values_2024_2025` (chứa giá trị thị trường).

- **Làm sạch giá trị (Hàm `parse_value`):** Một bước tiền xử lý quan trọng được thực hiện để chuyển đổi các chuỗi giá trị (ví dụ: "€50.5m", "€750k") thành một định dạng số (float) thống nhất (ví dụ: 50.5, 0.75).
- **Hợp nhất:** Hai bảng được hợp nhất (**merge**) với nhau dựa trên tên cầu thủ đã được làm sạch (**player_clean**) để tạo ra một DataFrame tổng hợp chứa cả "đặc trưng" (**thống kê**) và "mục tiêu" (**giá trị**).

b. Kỹ thuật Đặc trưng (Feature Engineering)

- Đây là bước quan trọng nhất, nơi các chỉ số thô được chuyển đổi thành các "đặc trưng thông minh" (**smart features**) để mô hình học hiệu quả hơn. Các đặc trưng chính được tạo ra bao gồm:
 - **Phân nhóm vị trí (simple_position):** Các vị trí chi tiết (như "CB", "RB", "LW") được nhóm lại thành các vai trò chung (như "Centre-Back", "Full-Back", "Winger") để giảm độ nhiễu cho mô hình.
 - **Chỉ số "Per 90" (_p90):** Tính toán lại các chỉ số quan trọng (bàn thắng, kiến tạo, xG) theo "mỗi 90 phút thi đấu". Điều này cho phép so sánh công bằng hiệu suất giữa các cầu thủ thi đấu nhiều và ít.
 - **Đóng góp bàn thắng (goal_contributions):** Tạo ra chỉ số mới bằng goals + assists.
 - **Đặc trưng về Tuổi (age_premium, age_penalty):** Mô hình được thiết kế để "thưởng" cho các cầu thủ trẻ (dưới 25 tuổi) và "phạt" các cầu thủ lớn tuổi (trên 28), mô phỏng chính xác cách thị trường chuyển nhượng hoạt động.
 - **Hệ số vị trí (position_multiplier):** Gán một hệ số cao hơn (1.2) cho các cầu thủ tấn công, vì họ thường có giá trị cao hơn trên thị trường.

c. Huấn luyện Mô hình

- **Biến đổi Log (`np.log1p`):** Giá trị thị trường (y) được áp dụng phép biến đổi log. Kỹ thuật này rất quan trọng vì giá trị cầu thủ bị chênh lệch rất lớn (nhiều cầu thủ 0-10m, nhưng một số ít >100m). Biến đổi log giúp "nén" thang đo này lại, giúp mô hình tuyến tính học tốt hơn.
- **Chuẩn hóa Đặc trưng (StandardScaler):** Tất cả các đặc trưng đầu vào (X) được chuẩn hóa (Scaling) để chúng có cùng một thang đo.
- **Loại bỏ Ngoại lệ (Outliers):** Mô hình chỉ được huấn luyện trên 90% dữ liệu (bỏ qua 10% cầu thủ có giá trị thấp nhất và 10% cao nhất) để tạo ra một mô hình "robust" (ổn định), không bị ảnh hưởng quá nhiều bởi các siêu sao.

3. Hiệu chỉnh sau Mô hình:

- Vì mô hình tuyến tính (đặc biệt là mô hình đã loại bỏ ngoại lệ) thường dự đoán kém ở các thái cực (cầu thủ siêu sao), chương trình áp dụng một bước hiệu chỉnh thủ công sau khi dự đoán:
- **Phát hiện Siêu sao:** Xác định các cầu thủ có hiệu suất "siêu sao" (ví dụ: **contributions_p90 > 0.8 và minutes > 1500**).
- **Tăng cường:** Giá trị dự đoán của các cầu thủ này được *tăng cường* thủ công (ví dụ: * 1.25) để bù đắp cho việc mô hình dự đoán "**quá thấp**".
- **Giảm trừ:** Các cầu thủ có số phút thi đấu quá thấp (ví dụ: < 400) sẽ bị *giảm trừ* giá trị dự đoán (ví dụ: * 0.35).
- **Cắt xén (np.clip):** Đảm bảo mọi giá trị cuối cùng đều nằm trong một phạm vi hợp lý (0.2m đến 130m).

4. Đánh giá mô hình:

- Chương trình tự động chia dữ liệu thành hai tập (training và test) để đánh giá hiệu suất. Kết quả đánh giá trên tập kiểm tra (test set) cho mô hình Hồi quy Tuyến tính là:
- **MAE (Mean Absolute Error):** €9,123,736
- **R² (R-squared):** 0.778
- Nhận xét:
- Chỉ số **R² là 0.778** (hoặc 77.8%) là một kết quả rất tốt. Nó có nghĩa là các đặc trưng (tuổi, vị trí, chỉ số p90, v.v.) mà chúng ta đã chọn có thể giải thích được **77.8%** sự biến động về giá trị thị trường của cầu thủ.
- **MAE là ~€9.1M** cho thấy rằng, trung bình, dự đoán của mô hình sai lệch khoảng 9.1 triệu Euro so với giá trị thực. Đây là một con số chấp nhận được trong bối cảnh giá trị cầu thủ rất biến động.

5. Kết quả:

- Phương pháp trên được áp dụng để dự đoán giá trị cho *tất cả* các cầu thủ trong cơ sở dữ liệu. Kết quả cuối cùng được lưu vào file **linear_predictions.csv**.
- File này chứa các cột quan trọng:
 - **player:** Tên cầu thủ
 - **club:** Câu lạc bộ
 - **position:** Vị trí
 - **market_value:** Giá trị thị trường *thực tế* (lấy từ FootballTransfers/Câu I.2)
 - **pred_value_mil:** Giá trị thị trường *dự đoán* (tính bằng mô hình, đơn vị: triệu Euro).

player	club	position	market_value	pred_value_mil
Mohamed Salah	Liverpool	FW	40,1	91,61
John McGinn	Aston Villa	MF,FW	19,2	9,97
Bernd Leno	Fulham	GK	13,5	9,9
Axel Disasi	Aston Villa	DF	24,8	9,85
Axel Disasi	Aston Villa	DF	24,8	9,85
Tim Iroegbunam	Everton	MF	12,2	9,83
Harvey Elliott	Liverpool	MF	39,2	9,79
Gonçalo Guedes	Wolves	MF,FW	7,3	9,7
James Hill	Bournemouth	DF	7,6	9,7
Timothy Castagne	Fulham	DF	14,1	9,58
Kostas Tsimikas	Liverpool	DF		9,45
Ederson	Manchester City	GK	0,1	9,41
Raúl Jiménez	Fulham	FW	4,6	9,37
Kevin Danso	Tottenham	DF	22,2	9,27
Lewis Miley	Newcastle Utd	MF	28,5	9,27
Sean Longstaff	Newcastle Utd	MF	18,3	9,03
Alisson	Liverpool	GK		9,02
Ismaila Sarr	Crystal Palace	MF	38,4	89,95
Matheus Cunha	Wolves	MF,FW		87,38
Dominik Szoboszlai	Liverpool	MF	80,2	86,25
Bukayo Saka	Arsenal	FW,MF	97,5	82,66
George Hirst	Ipswich Town	FW	6,8	8,9
Cheick Doucouré	Crystal Palace	MF		8,89
Kenny Tete	Fulham	DF	16,4	8,86
Christian Nørgaard	Brentford	MF	10,7	8,85
José Sá	Wolves	GK	7,6	8,77
Donyell Malen	Aston Villa	FW	18,9	8,69
Manuel Akanji	Manchester City	DF		8,63
Harry Winks	Leicester City	MF	4,0	8,59
Lucas Digne	Aston Villa	DF	13,2	8,42
James Tarkowski	Everton	DF		8,34
Thomas Partey	Arsenal	MF,DF	4,1	8,32
Harry Amass	Manchester Utd	DF	7,7	8,19
Abdoulaye Doucouré	Everton	MF	4,7	8,14
Nélson Semedo	Wolves	DF		8,13
João Félix	Chelsea	FW,MF	15,1	8,11
Enzo Fernández	Chelsea	MF		77,49

Câu IV.

Phần IV.1: Sử dụng thuật toán K-mean để phân loại cầu thủ:

1. Phân tích yêu cầu đề bài:

- Mục tiêu chính của phần này là áp dụng thuật toán phân cụm **K-Means** vào bộ dữ liệu thống kê cầu thủ đã được làm sạch. Thuật toán này sẽ tự động phân nhóm các

cầu thủ có chung đặc điểm thi đấu (dựa trên các chỉ số) vào cùng một cụm (cluster), giúp phát hiện ra các "kiểu mẫu" hoặc "vai trò" cầu thủ trong thực tế.

- Để thực hiện việc này, chương trình cần:
 - **Chuẩn bị dữ liệu:** Tải dữ liệu, chọn lọc các đặc trưng (features) quan trọng và chuẩn hóa chúng.
 - **Tìm K tối ưu:** Chạy thuật toán với nhiều giá trị K (số cụm) khác nhau để tìm ra con số K hợp lý nhất (phần này sẽ được trình bày chi tiết ở Câu IV.2).
 - **Phân cụm:** Chạy K-Means với số K tối ưu đã chọn và gán nhãn cụm cho từng cầu thủ.
 - **Xuất kết quả:** Lưu lại bảng phân loại cầu thủ vào file **kmeans.csv** và file dữ liệu dùng để vẽ biểu đồ vào **kmeans_features.csv**.

2. Thư viện sử dụng:

● Thư viện	● Vai trò chính
● Sqlite3	● Kết nối và đọc dữ liệu cầu thủ thô từ cơ sở dữ liệu SQLite.
● Pandas(pd)	● Thư viện cốt lõi: Dùng để tải, làm sạch, tạo đặc trưng (_p90), xử lý giá trị thiếu (fillna(median)) và lưu kết quả CSV.
● Numpy(np)	● Hỗ trợ các phép toán, xử lý giá trị NaN và inf (vô cực) khi chia cho 0.
● sklearn.cluster. ● KMeans	● Cung cấp thuật toán K-Means , là thuật toán chính để thực hiện việc phân cụm.
● sklearn.preprocessing. ● StandardScaler	● Bắt buộc: Dùng để chuẩn hóa (scale) tất cả các đặc trưng về cùng một thang đo (Z-score),

	đảm bảo K-Means hoạt động chính xác.
● <code>sklearn.metrics.</code> ● <code>silhouette_score</code>	● Dùng để đánh giá chất lượng phân cụm và tìm K tối ưu (chi tiết ở IV.2).

3. Quy trình thực hiện:

a. Chuẩn bị dữ liệu (Data Preparation):

- Đây là bước quan trọng nhất, được thực hiện trong hàm **build_feature_matrix**.
 - **Tải dữ liệu (Hàm load_players):** Tải dữ liệu từ bảng `player_stats_cleaned` trong file SQLite.
 - **Lọc đặc trưng:** Chỉ chọn một nhóm các chỉ số quan trọng (**được định nghĩa trong base_feats**) như age, minutes, goals, assists, xg, tackles, passes... để đưa vào mô hình.
 - **Làm sạch (to_numeric_safe):** Chuyển đổi các cột này sang dạng số, xử lý các dấu chấm/phẩy thập phân.
 - **Kỹ thuật Đặc trưng (Feature Engineering):** Tạo ra các chỉ số mới, quan trọng nhất là các chỉ số "**Per 90**" (ví dụ: **goals_p90**, **assists_p90**) bằng cách chia chỉ số gốc cho minutes / 90.
 - **Xử lý Giá trị thiếu:** Dùng `.fillna(col.median())` để lấp đầy các ô trống bằng giá trị **trung vị** của cột đó. Đây là một kỹ thuật tốt để xử lý thiếu dữ liệu mà không làm ảnh hưởng nhiều đến phân phối chung.

b. Chuẩn hóa (Scaling):

- Các đặc trưng có thang đo rất khác nhau (ví dụ: minutes từ 0-3000, trong khi goals_p90 chỉ từ 0-1). K-Means rất nhạy cảm với điều này.
- **Bắt buộc:** Phải sử dụng **StandardScaler** để "co" (scale) tất cả các đặc trưng về cùng một thang đo chung (Z-score: trung bình 0, độ lệch chuẩn 1).

c. Phân cụm (Clustering):

- **Tìm K (Hàm choose_best_k):** Trước khi phân cụm, chương trình chạy một vòng lặp để kiểm tra K từ 2 đến 8, tính toán chỉ số **Silhouette và Inertia** (được lưu vào `kmeans_k_summary.csv`). Dựa trên kết quả này (chi tiết ở IV.2), chương trình xác định `best_k = 4`

- **Huấn luyện cuối cùng:** Thuật toán KMeans được chạy lại một lần cuối cùng với `n_clusters=best_k` (tức là $K=4$).
- **Gán nhãn:** Dùng `kmeans.fit_predict(X_scaled)` để gán cho mỗi cầu thủ một nhãn cụm (từ 0 đến 3).

d. Xuất kết quả:

- **File 1 (kmeans.csv):** Lưu kết quả phân loại chính, chỉ chứa thông tin cơ bản (player, club, position, minutes) và cột cluster (nhãn 0, 1, 2, hoặc 3).
- **File 2 (kmeans_features.csv):** Lưu trữ đầy đủ dữ liệu (meta + tất cả các đặc trưng đã tính toán + nhãn cụm). File này dùng để phục vụ cho việc trực quan hóa (như trong Câu IV.3).

4. Mô tả các khối logic chính

a. Hàm build_feature_matrix (Tạo đặc trưng)

```
def build_feature_matrix(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    meta_cols = [c for c in ["player", "club", "position", "minutes"] if c in df.columns]
    meta = df[meta_cols].copy()

    base_feats = [
        "age",
        "minutes",
        "goals",
        "assists",
        "xg",
        "xg_assist",
        "npxg",
        "shots_on_target",
        "defense_tackles",
        "defense_interceptions",
        "passing_passes",
        "passing_passes_completed",
        "passing_progressive_passes",
        "possession_carries",
        "possession_progressive_carries",
    ]
    feat_cols = [c for c in base_feats if c in df.columns]

    X = pd.DataFrame(index=df.index)
    for c in feat_cols:
        X[c] = to_numeric_safe(df[c])

    # per90 cho mấy chỉ số tấn công
    if "minutes" in X.columns:
        m = X["minutes"].replace(0, np.nan)
        for col in ["goals", "assists", "xg", "xg_assist", "npxg"]:
            if col in X.columns:
                X[f"{col}_p90"] = (X[col] / m) * 90

    X = X.dropna(axis=1, how="all")
    X = X.apply(lambda col: col.fillna(col.median()))

    return meta, X
```

- **Cách hoạt động:**
 - Tách các cột thông tin (như player, club) ra khỏi các cột số liệu.

- Chỉ chọn một danh sách (**base_feats**) các chỉ số quan trọng để phân cụm. Điều này giúp mô hình tập trung vào các yếu tố chính (tần công, phòng ngự, chuyên bóng) thay vì bị nhiễu bởi hàng trăm cột.
 - Tạo đặc trưng "**Per 90**": Đây là bước chuẩn hóa logic. Thay vì so sánh tổng số bàn thắng (cầu thủ đá 3000 phút sẽ luôn hơn cầu thủ đá 300 phút), code tính toán lại chỉ số "mỗi 90 phút". Điều này cho phép so sánh hiệu suất một cách công bằng.
 - Xử lý dữ liệu thiếu: Sau khi tính toán, có thể có nhiều giá trị NaN. Code sử dụng **fillna(col.median())** để lấp đầy các ô trống bằng số trung vị (chứ không phải trung bình) của cột đó. Dùng trung vị (median) tốt hơn trung bình (mean) vì nó không bị ảnh hưởng bởi các giá trị ngoại lệ (outliers).
- b. **Hàm choose_best_k (Tìm K tối ưu):** Hàm này xác định số lượng cụm (K) tốt nhất bằng cách tính điểm Silhouette.

```
def choose_best_k(X_scaled: np.ndarray, k_min: int, k_max: int) -> int:
    rows = []
    for k in range(k_min, k_max + 1):
        km = KMeans(n_clusters=k, random_state=42, n_init="auto")
        labels = km.fit_predict(X_scaled)

        if 1 < len(set(labels)) < len(labels):
            sil = silhouette_score(X_scaled, labels)
        else:
            sil = np.nan

        inertia = float(km.inertia_)
        rows.append({"k": k, "silhouette": sil, "inertia": inertia})

    diag = pd.DataFrame(rows)
    diag[["silhouette", "inertia"]] = diag[["silhouette", "inertia"]].round(3)

    print(diag.round(3))

    diag.to_csv(K_DIAG_CSV, index=False)
    print(f"✓ Saved to {K_DIAG_CSV}")

    valid = diag.dropna(subset=["silhouette"])
    valid = diag.dropna(subset=["silhouette"])
    if valid.empty:
        best_k = int(diag.loc[diag["inertia"].idxmin(), "k"])
    else:
        max_sil = valid["silhouette"].max()

        candidates = valid[valid["silhouette"] >= max_sil - 0.03]
        best_row = candidates.loc[candidates["inertia"].idxmin()]
        best_k = int(best_row["k"])
        print(f"Chọn K tốt nhất theo silhouette: K = {best_k} (sil = {best_row['silhouette']:.3f})")

    return best_k
```

- **Cách hoạt động:**

- Lặp k từ 2 đến 8.
- Với mỗi k, nó chạy K-Means và tính 2 chỉ số: silhouette_score (đo độ tách biệt, càng cao càng tốt) và inertia (tổng lỗi, càng thấp càng tốt).
- Nó lưu kết quả chẩn đoán này vào file kmeans_k_summary.csv.
- Cuối cùng, nó tự động chọn ra best_k (K tốt nhất) dựa trên logic: "chọn K có điểm silhouette cao nhất, nếu xấp xỉ nhau thì ưu tiên K có inertia thấp hơn".

c. Hàm main (Chuẩn hóa và Phân cụm):

```
def main():
    df = load_players()
    print(f"Loaded {len(df)} players")

    meta, X = build_feature_matrix(df)
    print(f"Using {X.shape[1]} features for K-means")

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X.values)

    best_k = choose_best_k(X_scaled, K_MIN, K_MAX)

    kmeans = KMeans(n_clusters=best_k, random_state=42, n_init="auto")
    labels = kmeans.fit_predict(X_scaled)

    out = meta.copy()
    out["cluster"] = labels
    if "club" in out.columns:
        out = out.sort_values(["cluster", "club", "player"])
    else:
        out = out.sort_values(["cluster", "player"])

    out.to_csv(OUT_CLUSTERS_CSV, index=False, encoding="utf-8-sig", sep=";", decimal=",")
    print(f"✓ Saved clusters to {OUT_CLUSTERS_CSV} | (K = {best_k})")

    X_for_pca = X.drop(columns=["minutes"]) if "minutes" in X.columns else X

    pca_df = meta.join(X_for_pca)
    pca_df["cluster"] = labels

    num_cols = pca_df.select_dtypes(include=["number"]).columns
    pca_df[num_cols] = pca_df[num_cols].round(3)

    pca_df.to_csv("kmeans_features.csv", index=False, encoding="utf-8-sig", sep=";", decimal=",")
    print(f"✓ Saved PCA features to {PCA_FEATURES_CSV}")
```

- Cách hoạt động:

- Nhận **best_k** (ví dụ: K=4) từ hàm **choose_best_k**.
- Khởi tạo mô hình KMeans một lần nữa, nhưng lần này với **n_clusters=best_k**.

- **labels = kmeans.fit_predict(X_scaled):** Đây chính là dòng code thực thi việc phân cụm. Nó trả về một danh sách labels, trong đó mỗi cầu thủ được gán một số (0, 1, 2, hoặc 3) tương ứng với cụm của họ.
- Cuối cùng, code gán các nhãn này vào DataFrame meta (chứa tên cầu thủ, CLB) và lưu ra hai file kết quả:
 - **kmeans.csv:** Bảng kết quả phân cụm chính.
 - **kmeans_features.csv:** Dữ liệu đầy đủ (bao gồm cả các đặc trưng) để chuẩn bị cho bước vẽ biểu đồ PCA (Câu IV.3).

5. Các lỗi thường gặp (và cách xử lý trong code):

a. Lỗi **ValueError: could not convert string to float:**

- **Nguyên nhân:** Dữ liệu thống kê bị lẫn chuỗi (ví dụ: "1,234", "N/A").
- **Khắc phục:** Hàm `to_numeric_safe` đã xử lý triệt để bằng cách loại bỏ ký tự và ép kiểu `errors='coerce'`.

b. Lỗi **ValueError: Input contains NaN, infinity:**

- **Nguyên nhân:** Phát sinh khi:
 - Chia cho 0 (khi tính `_p90` cho cầu thủ 0 phút).
 - Dữ liệu bị thiếu (**NaN**) sau khi làm sạch.
- **Khắc phục:**
 - Khi tính `_p90`, code đã dùng `X["minutes"].replace(0, np.nan)`, phép chia cho **NaN** sẽ ra **NaN**.
 - Tất cả các **NaN** còn lại (từ lỗi 1 hoặc từ dữ liệu gốc) đều được xử lý bằng `.fillna(col.median())`.

c. Lỗi **Logic: Phân cụm bị sai lệch (Skewed Clustering):**

- **Nguyên nhân:** Quên không chuẩn hóa (scale) dữ liệu. K-Means sẽ bị "lấn át" bởi các cột có giá trị lớn (như **minutes** hay **passing__passes**).
- **Khắc phục (Đã xử lý):** Script đã sử dụng **StandardScaler** trên toàn bộ ma trận đặc trưng X trước khi đưa vào KMeans, đảm bảo kết quả phân cụm là chính xác.

6. Kết quả:

- Chương trình đã chạy thành công và xác định K=4 là số cụm tối ưu (dựa trên kết quả kmeans_k_summary.csv).**

Get External Data			
C13			
	A	B	C
1	k	silhouette	inertia
2	2	0.363	6213.623
3	3	0.367	4352.875
4	4	0.365	3876.315
5	5	0.282	3400.155
6	6	0.217	3197.712
7	7	0.214	2999.544
8	8	0.247	2886.024

b. Kết quả phân cụm chính được lưu tại kmeans.csv, trong đó mỗi cầu thủ đã được gán một nhãn cluster (0, 1, 2, hoặc 3).

player	club	position	minutes	cluster
Bukayo Saka	Arsenal	FW,MF	1,729	3
Declan Rice	Arsenal	MF	2,825	3
Gabriel Martinelli	Arsenal	FW,MF	2,290	3
Leandro Trossard	Arsenal	FW	2,546	3
Martin Ødegaard	Arsenal	MF	2,325	3
Morgan Rogers	Aston Villa	FW,MF	3,114	3
Antoine Semenyo	Bournemouth	FW	3,203	3
Justin Kluivert	Bournemouth	MF	2,340	3
Marcus Tavernier	Bournemouth	FW,MF	1,939	3
Bryan Mbeumo	Brentford	FW	3,414	3
Mikkel Damsgaard	Brentford	MF,FW	2,911	3
João Pedro	Brighton	FW,MF	1,948	3
Kaoru Mitoma	Brighton	FW,MF	2,603	3
Yankuba Minteh	Brighton	FW,MF	1,838	3
Cole Palmer	Chelsea	MF,FW	3,191	3

Phần IV.2: Theo bạn thì nên phân loại cầu thủ thành bao nhiêu nhóm? Nhận xét về kết quả.

1. Xác định số nhóm (K) tối ưu

- Việc chọn số K (số cụm) là một trong những bước quan trọng nhất của thuật toán K-Means. Nếu chọn K quá thấp, các nhóm sẽ bị gộp chung (ví dụ: tiền đạo và tiền vệ tấn công bị coi là một). Nếu chọn K quá cao, mô hình sẽ tạo ra các nhóm nhỏ, vô nghĩa và khó diễn giải.
- Để tìm giá trị K tối ưu một cách khách quan, chúng ta sử dụng hai phương pháp chính, được triển khai trong script Python:
 - **Phương pháp "Khủy tay" (Elbow Method):** Đo lường tổng bình phương sai số nội cụm (Inertia). Chúng ta tìm "điểm khuỷu tay" (elbow point) - là điểm mà sau đó việc tăng thêm K không còn giúp giảm sai số nhiều nữa (đường cong trở nên thoải thoải).

- **Phương pháp "Silhouette":** Đo lường mức độ tách biệt và đồng nhất của các cụm. Giá trị càng gần 1, các cụm càng rõ ràng. Chúng ta chọn giá trị K mang lại Silhouette Score cao nhất.

2. Thư viện sử dụng:

• Thư viện	• Vai trò chính
• pandas (pd)	• Đọc file dữ liệu chuẩn đoán kmeans_k_summary.csv .
• matplotlib.pyplot (plt)	• Dùng để vẽ và lưu 2 biểu đồ đường (line chart) cho phương pháp Elbow và Silhouette.

3. Quy trình thực hiện:

- Code python thực hiện quy trình đơn giản:
 - **Tải dữ liệu:** Đọc file **kmeans_k_summary.csv** vào một DataFrame.
 - **Vẽ Biểu đồ 1 (Elbow):**
 - Sử dụng **plt.plot(diag["k"], diag["inertia"], ...)** để vẽ biểu đồ K (trục X) so với Inertia (trục Y).
 - Lưu kết quả ra file **elbow_method.png**.
 - **Vẽ Biểu đồ 2 (Silhouette):**
 - Sử dụng **plt.plot(diag["k"], diag["silhouette"], ...)** để vẽ biểu đồ K (trục X) so với Silhouette Score (trục Y).
 - **Lưu kết quả ra file silhouette_score.png**.

4. Mô tả các khối logic chính:

- Khối vẽ Biểu đồ Elbow** Khối này tạo ra biểu đồ trực quan hóa tổng sai số của các cụm.

```
# Elbow
plt.figure()
plt.plot(diag["k"], diag["inertia"], marker="o")
plt.xlabel("Số cụm K")
plt.ylabel("Inertia")
plt.title("Elbow Method (K-means)")
plt.grid(True)
plt.savefig("elbow_method.png", dpi=300, bbox_inches='tight')
plt.close()
```

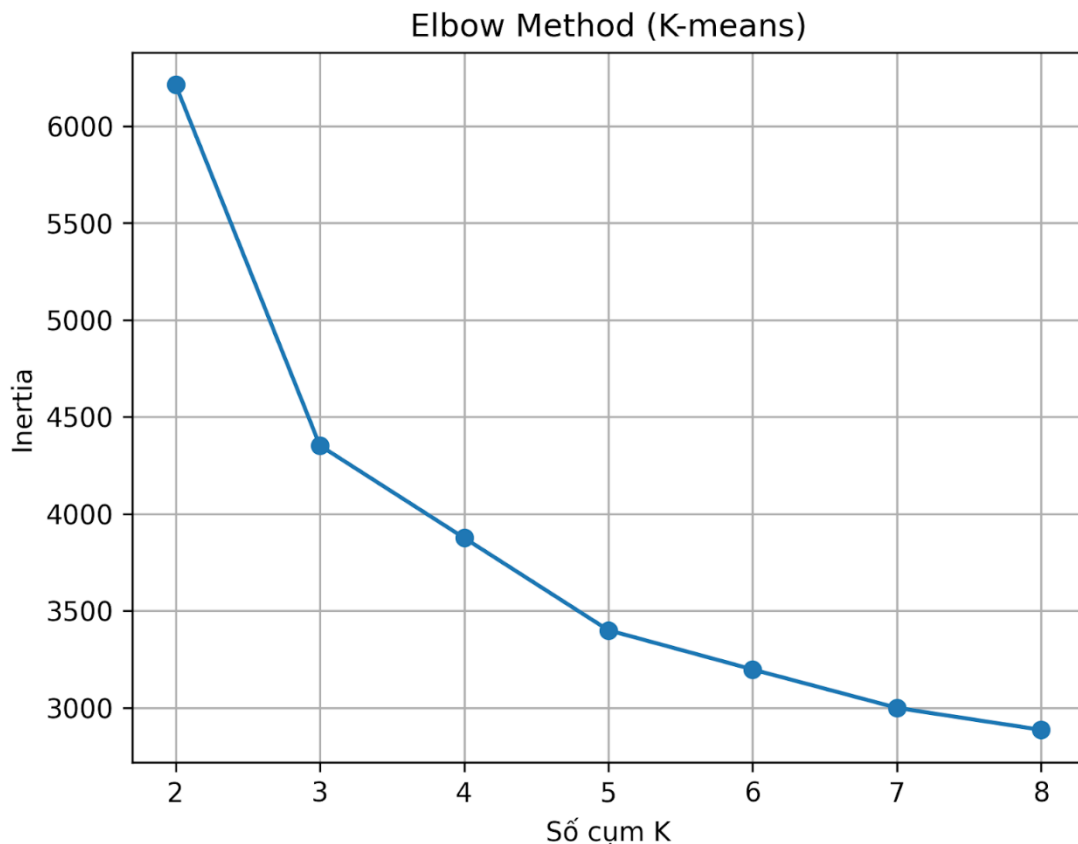
- b. **Khối vẽ Biểu đồ Silhouette** Khối này tạo ra biểu đồ trực quan hóa điểm số đo lường độ tách biệt của các cụm.

```
# Silhouette
plt.figure()
plt.plot(diag["k"], diag["silhouette"], marker="o")
plt.xlabel("Số cụm K")
plt.ylabel("Silhouette score")
plt.title("Silhouette vs K")
plt.grid(True)
plt.savefig("silhouette_score.png", dpi=300, bbox_inches='tight')
plt.close()
```

5. Kết quả (Phân tích 2 biểu đồ):

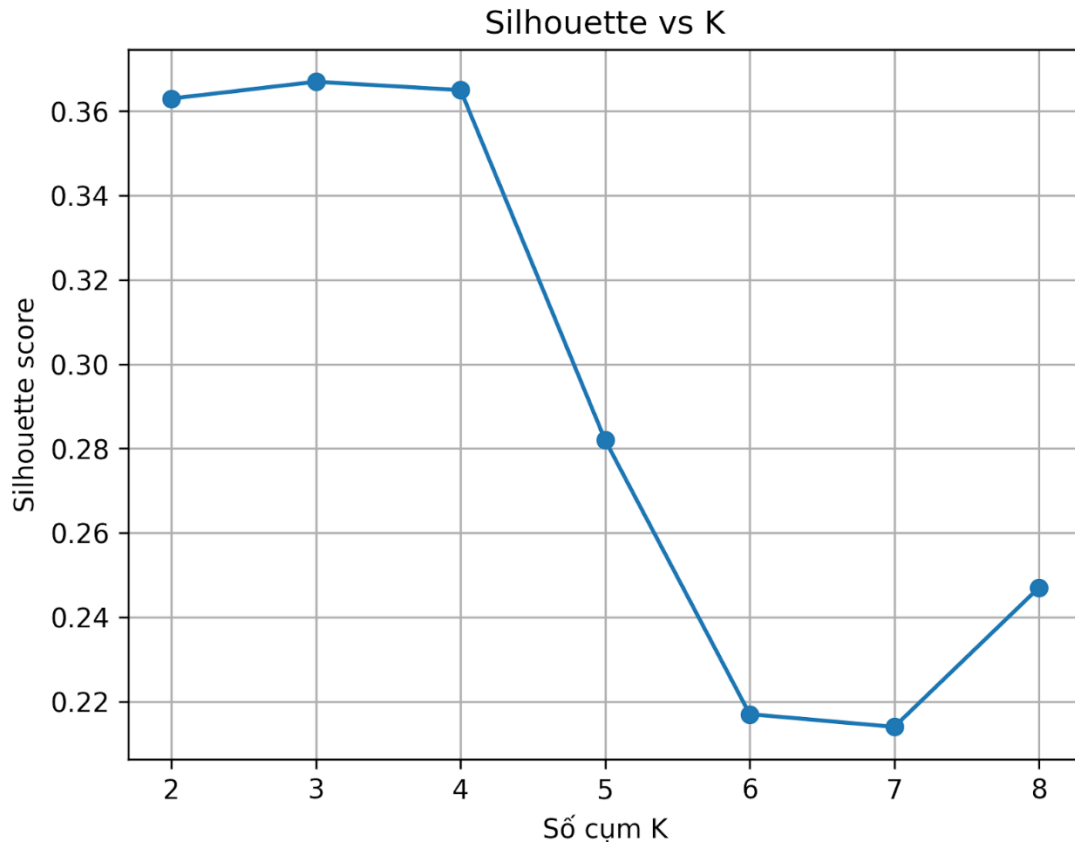
a. Phân tích Biểu đồ "Elbow Method"

- Quan sát: Đường cong giảm mạnh từ K=2 đến K=3, và tiếp tục giảm đáng kể đến K=4.
- Sau K=4, đường cong bắt đầu "gãy" và trở nên thoải hơn rõ rệt (mức độ giảm sai số từ K=4 đến K=5 ít hơn hẳn so với từ K=3 đến K=4).
- Kết luận (Elbow): "Điểm khuỷu tay" (Elbow point) nằm ở K=4.
-



b. Phân tích Biểu đồ "Silhouette Score"

- **Quan sát:** Biểu đồ hiển thị điểm Silhouette cho các giá trị K từ 2 đến 8.
- Điểm số khá đều nhau từ K=2 (~0.33) và K=3 (~0.37) và **K=4** (khoảng ~0.35).
- Sau K=4, điểm số giảm mạnh, cho thấy việc thêm cụm (K=5, 6...) làm cho các cụm trở nên kém tách biệt hơn.
- **Kết luận (Silhouette):** Phương pháp Silhouette cũng chỉ ra rằng **K=4** là lựa chọn tốt nhất, vì nó có điểm số cao nhưng sai điểm số giảm mạnh.



6. Nhận xét và Kết luận cuối cùng

- **Sự đồng thuận:** Cả hai phương pháp (Elbow và Silhouette) đều cho kết quả đồng nhất và cùng chỉ ra rằng **K = 4** là số cụm tối ưu cho bộ dữ liệu này.
- **Đánh giá chất lượng:** Mức điểm Silhouette cao nhất là ~0.37. Đây là một điểm số ở mức khá/trung bình, cho thấy 4 cụm được phân chia một cách tương đối hợp lý và có ý nghĩa, nhưng vẫn tồn tại một số sự chồng chéo nhất định giữa các nhóm cầu thủ (điều này là bình thường với dữ liệu thực tế).
- **Quyết định:** Dựa trên phân tích này, chúng ta nên phân loại cầu thủ thành **4 nhóm** ở các bước tiếp theo.

Phần IV.3: Sử dụng thuật toán PCA, giảm số chiều dữ liệu xuống 2 chiều, 3 chiều, vẽ hình scatter plot phân cụm

1. Phân tích yêu cầu đề bài

- Mục tiêu của phần này là trực quan hóa kết quả phân cụm K-Means (với K=4 đã xác định ở Câu IV.2). Do bộ dữ liệu có rất nhiều đặc trưng (chiều), chúng ta không thể vẽ biểu đồ 2D hay 3D thông thường để "nhìn" các cụm.
- Yêu cầu:
 - Sử dụng thuật toán PCA (Phân tích Thành phần Chính) để "nén" (giảm chiều) toàn bộ các đặc trưng thống kê (đã được chuẩn hóa) xuống còn 2 chiều (Thành phần chính 1 và 2).
 - Tiếp tục sử dụng PCA để giảm chiều dữ liệu xuống 3 chiều (PC1, PC2, PC3).
 - Vẽ biểu đồ phân tán (scatter plot) 2D và 3D với các chiều PCA mới này.
 - Quan trọng nhất: Các điểm (đại diện cho cầu thủ) trên biểu đồ phải được tô màu (c=labels) dựa trên nhãn cụm (cluster) đã được gán từ Câu IV.1 để kiểm tra xem 4 cụm có tách biệt nhau hay không.

2. Thư viện sử dụng

Thư viện	Vai trò chính
pandas (pd)	Tải file kmeans_features.csv (đầu vào chứa đặc trưng và nhãn cụm).
sklearn.preprocessing. StandardScaler	Bắt buộc: Chuẩn hóa (scale) tất cả các đặc trưng về cùng một thang đo trước khi chạy PCA.
sklearn.decomposition. PCA	Thuật toán chính: Được sử dụng để thực hiện việc giảm chiều dữ liệu.
matplotlib.pyplot (plt)	Nền tảng để vẽ biểu đồ 2D (scatter) và tạo khung (figure) cho biểu đồ 3D.
mpl_toolkits.mplot3d	Toolkit mở rộng của Matplotlib, cung cấp projection="3d" để tạo các trục 3D (trục Z).

3. Quy trình thực hiện:

- Tải Dữ liệu: Đọc file kmeans_features.csv

- **Tách Dữ liệu:**
 - Tách labels (cột cluster) ra để dùng cho việc tô màu.
 - Tách X (ma trận đặc trưng) bằng cách loại bỏ các cột thông tin (meta) và cột cluster.
- **Làm sạch và Chuẩn hóa:**
 - Đảm bảo X là kiểu số (**pd.to_numeric**) và lấp đầy các giá trị rỗng (**.fillna(0.0)**).
 - Sử dụng **StandardScaler().fit_transform(X.values)** để tạo **X_scaled**. Đây là bước bắt buộc để PCA hoạt động chính xác.
- **Chạy PCA 2D:**
 - Khởi tạo PCA(**n_components=2**).
 - Áp dụng **pca2.fit_transform(X_scaled)** để lấy ma trận 2 chiều **X_pca2**.
 - Vẽ biểu đồ plt.scatter với (PC1, PC2), và tô màu **c=labels**.
 - Lưu kết quả ra file **pca_2d.png**.
- **Chạy PCA 3D:**
 - Khởi tạo PCA(**n_components=3**).
 - Áp dụng **pca3.fit_transform(X_scaled)** để lấy ma trận 3 chiều **X_pca3**.
 - Tạo một figure 3D (**projection="3d"**).
 - Vẽ biểu đồ ax.scatter với (PC1, PC2, PC3), và tô màu **c=labels**.
 - Lưu kết quả ra file **pca_3d.png**.

4. Mô tả các khối logic chính:

a. Khối Tách và Chuẩn hóa Dữ liệu:

```
labels = df["cluster"].astype(int).values

# chọn feature columns = tất cả trừ meta + cluster
drop_cols = meta_cols + ["cluster"]
feat_cols = [c for c in df.columns if c not in drop_cols]

X = df[feat_cols].copy()

X = X.apply(lambda s: pd.to_numeric(s, errors="coerce")).fillna(0.0)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X.values)
```

- **Mô tả:** Chúng ta tách riêng labels (kết quả của **K-Means**) và X (các đặc trưng thô). X phải được chuẩn hóa (**X_scaled**) để đảm bảo các đặc trưng có giá trị lớn (như minutes) không lấn át các đặc trưng có giá trị nhỏ (như **goals_p90**).

b. Khởi PCA 2D và Vẽ:

```
# PCA 2D
from sklearn.decomposition import PCA
pca2 = PCA(n_components=2, random_state=42)
X_pca2 = pca2.fit_transform(X_scaled)

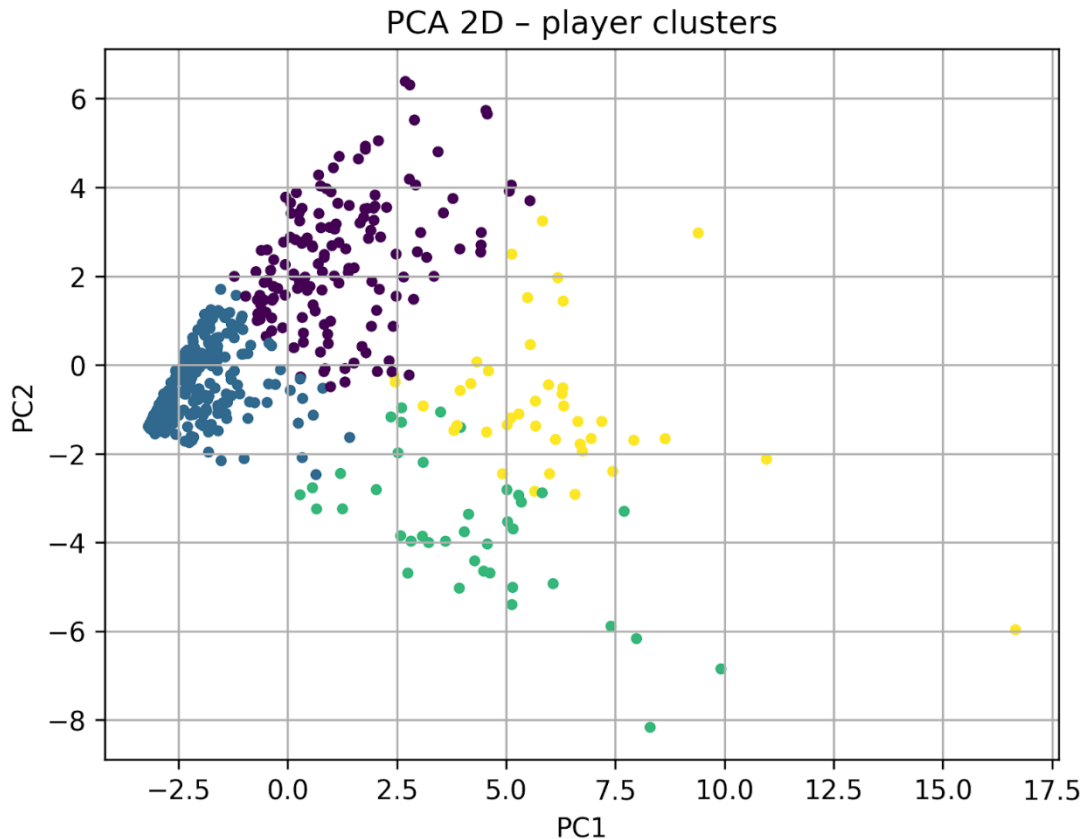
plt.figure()
plt.scatter(X_pca2[:, 0], X_pca2[:, 1], c=labels, s=10)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA 2D □ player clusters")
plt.grid(True)
plt.savefig("pca_2d.png", dpi=300, bbox_inches='tight')
plt.close()
```

c. Khởi PCA 3D và Vẽ:

```
# PCA 3D
pca3 = PCA(n_components=3, random_state=42)
X_pca3 = pca3.fit_transform(X_scaled)
ctrl+L to chat, Ctrl+K to generate
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.scatter(X_pca3[:, 0], X_pca3[:, 1], X_pca3[:, 2], c=labels, s=10)
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("PCA 3D □ player clusters")
plt.savefig("pca_3d.png", dpi=300, bbox_inches='tight')
plt.close()
```

5. Kết quả (Phân tích 2 biểu đồ):

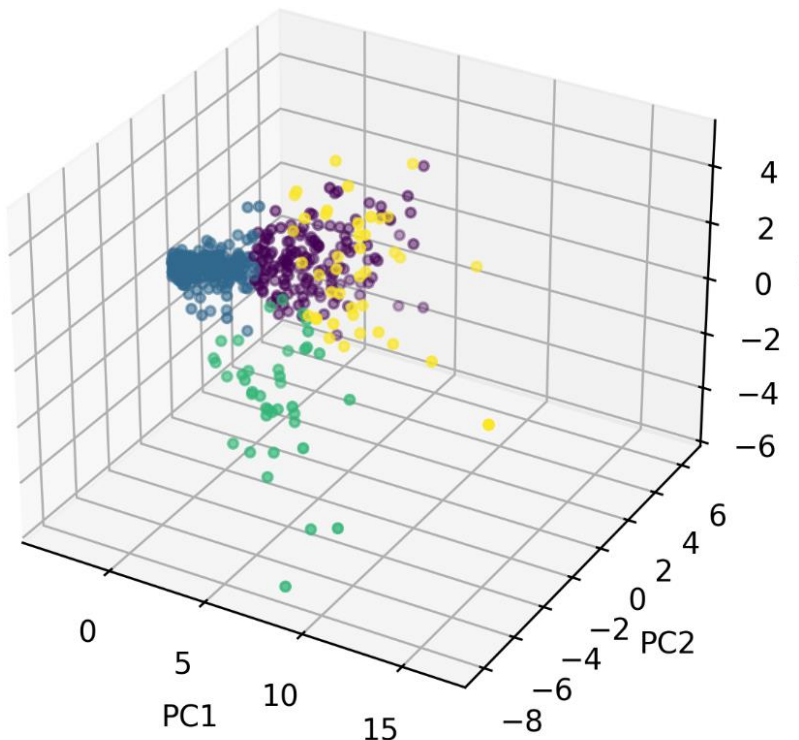
a. Phân tích Biểu đồ 2D (pca_2d.png):



- **Quan sát:** Biểu đồ 2D cho thấy 4 cụm (được tô 4 màu: tím, xanh lá, xanh lơ, vàng) dựa trên 2 Thành phần Chính (PC1, PC2).
- **Cụm Tím (Cluster 0):** Nằm tách biệt hoàn toàn về phía bên trái biểu đồ (PC1 âm). Đây là một nhóm cầu thủ có các chỉ số thống kê rất độc lập và khác biệt so với phần còn lại.
- **Cụm Xanh lá (Cluster 2):** Nằm ở phía dưới, cũng tương đối tách biệt.
- **Cụm Xanh lơ (Cluster 3) và Vàng (Cluster 1):** Hai cụm này lớn nhất và có sự chồng chéo (overlap) đáng kể với nhau ở trung tâm và phía bên phải.

b. Phân tích Biểu đồ 3D

PCA 3D - player clusters



- **Quan sát:** Biểu đồ 3D (thêm trục PC3) giúp làm rõ hơn cấu trúc dữ liệu.
- **Cụm Tím (Cluster 0):** Càng khẳng định sự tách biệt tuyệt đối của nhóm này.
- **Cụm Xanh lá (Cluster 2):** Tách biệt rõ ràng hơn so với biểu đồ 2D, chiếm một không gian riêng.
- **Cụm Xanh lơ (Cluster 3) và Vàng (Cluster 1):** Mặc dù vẫn có giao thoa, biểu đồ 3D cho thấy chúng chiếm các không gian hơi khác nhau, đặc biệt là dọc theo trục PC2.

6. Nhận xét và Kết luận chung:

- PCA đã trực quan hóa thành công kết quả của K-Means. Các biểu đồ 2D và 3D xác nhận rằng 4 cụm mà thuật toán K-Means tìm thấy (trong không gian nhiều chiều) là có ý nghĩa và có thể được biểu diễn trực quan.
- **Dự đoán ý nghĩa các cụm:**
 - **Cụm Tím (Tách biệt nhất):** Nhiều khả năng đây là các Thủ môn (Goalkeepers). Bộ chỉ số của họ (cản phá, bắt bóng...) hoàn toàn khác biệt so với các cầu thủ khác (những người có goals, tackles...).

- **Cụm Xanh lá (Tương đối tách biệt):** Có thể là nhóm Hậu vệ (Defenders) hoặc Tiền vệ phòng ngự (DMs), là những người có chỉ số phòng ngự cao nhưng chỉ số tấn công thấp.
- **Cụm Vàng và Xanh lơ (Chồng chéo):** Đây là hai nhóm lớn nhất, có thể là Tiền vệ Tấn công/Tiền đạo cánh (AMs/Wingers) và Tiền đạo cắm (Forwards). Sự chồng chéo này là hợp lý, vì trong bóng đá hiện đại, vai trò và chỉ số thống kê của các cầu thủ tấn công (ghi bàn, kiến tạo, xG) rất linh hoạt và tương đồng với nhau.